

Kyrian World: un planificador de viajes con IA generativa y recuperación aumentada

Proyecto Júpiter — Trabajo de Fin de Máster
Máster en Inteligencia Artificial, Cloud Computing y DevOps

Memoria técnica y de negocio: caso de uso, arquitectura, solución de IA, DevOps, evaluación y trabajo futuro

Manuel Pérez

Alexander De Sousa

Joaquín Castro Salas

David Baos

Versión 1.1, borrador para revisión — 23 de septiembre de 2026 — repositorio: github.com/manupm87/travel-ai-world

Resumen ejecutivo

Kyrian World es un planificador de viajes conversacional, Trabajo de Fin de Máster de un equipo de cuatro personas apoyado en agentes de IA, que construye itinerarios de una ciudad a partir de un corpus abierto con licencia limpia (Wikivoyage, Wikipedia, OpenStreetMap) mediante generación aumentada por recuperación. El MVP está desplegado en AWS en <https://kyrian-world.com>. La memoria plantea: ¿puede un RAG con anclaje estructural dar itinerarios verificables por poco más de 10 €/mes de coste fijo? En coste, sí; la calidad del itinerario y la aceptación por usuarios no están medidas. Toda tarjeta procede de un documento recuperado y los ids no recuperados se descartan; la prosa libre del modelo no está verificada. El coste fijo estimado es de 12–13 €/mes (≈5 € en Terraform más un WAF creado a mano; sin factura real todavía). S3 Vectors se eligió frente a Qdrant por simplicidad operativa y un *spike* lo confirmó. La viabilidad B2C es marginal; se recomienda el B2B para oficinas de turismo. La deuda principal es la falta de evaluación extremo a extremo, de alarmas y de límites de uso.

1 Introducción

Kyrian World es el Trabajo de Fin de Máster (Proyecto Júpiter) de un equipo de cuatro personas, iniciado el 10-03-2026 con un objetivo concreto: comprobar si un planificador de viajes con IA generativa puede ofrecer recomendaciones *verificables*, cada una anclada en un documento real, con un coste de operación de unos pocos euros al mes. Hasta el 23-09-2026 acumula 324 *commits* y 181 PR fusionadas, está desplegado en AWS (kyrian-world.com) y se ha desarrollado en buena parte con agentes de programación guiados por un proceso escrito (*brief*, implementación, revisión y corrección), con cada cambio trazable hasta un issue y, cuando procede, hasta una ADR.

Objetivos y alcance. Explicar las decisiones de producto, arquitectura y plataforma con su contexto y alternativas; valorar la viabilidad en su mercado; y cerrar con un análisis crítico y propuestas priorizadas. Se describe el sistema en el *commit* [e7916c8](#). Quedan fuera la alternativa en GCP, que no se despliega, y cualquier medida con usuarios reales, que no existen todavía.

Metodología. Los hechos proceden del repositorio: código, 23 ADR, *runbooks*, el documento del *spike* de vectores y el historial de *git* y de GitHub. Cada cifra indica si es *medida* (el *spike*; los tests, ejecutados con `just test` el 23-09-2026), *estimada* (costes de los ADR, economía unitaria) o *contada* (líneas, PR, documentos del corpus); lo no medido se declara. El estudio de mercado usa fuentes públicas citadas y marca sus proyecciones como hipótesis. La tabla 1 relaciona cada apartado y requisito técnico del guion con la sección que lo cubre.

Tabla 1: Correspondencia con el guion del Proyecto Júpiter.

Apartado o requisito del guion	Dónde	Estado
a) Introducción y resumen ejecutivo	Resumen, sección 1	cubierto
b) Caso de negocio y mercado	secciones 2 y 3	cubierto
c) Contribución individual	sección 4	cubierto
d) Arquitectura técnica	secciones 5 y 7	cubierto
e) Diseño de la solución de IA	sección 6	cubierto
f) Integración con DevOps	secciones 8 y 9	cubierto
g) Uso de modelos preentrenados	sección 6	cubierto
h) Evaluación de la solución	sección 11	parcial
i) Conclusiones y trabajo futuro	secciones 12 y 13	cubierto
IA generativa	sección 6	cubierto
Base de datos vectorial	sección 6	cubierto
MVP funcional	secciones 2 y 7	cubierto
API de modelo fundacional	sección 6	cubierto
Pipeline CI/CD	sección 9	cubierto
Monitorización y trazabilidad	sección 10	parcial

Estructura. Las secciones 2 y 3 presentan el producto, el mercado y la viabilidad, y la sección 4, el equipo y la contribución de cada miembro. Las secciones 5 a 7 describen las arquitecturas de software, de IA y de nube; las secciones 8 a 10 tratan el entorno de desarrollo, la entrega continua con agentes y la fiabilidad y los costes, y la sección 11, la evaluación. La sección 12 reúne la crítica y las propuestas, la sección 13 concluye y el anexo (Apéndice A) recoge un glosario y la tabla de ADR.

2 El producto

Kyrian World (kyrian-world.com) es un planificador de viajes conversacional que construye en tiempo real un itinerario día a día que se puede guardar y reabrir. El usuario objetivo, inferido del alcance (no hay estudio de usuarios), es el viajero individual en *una so-*

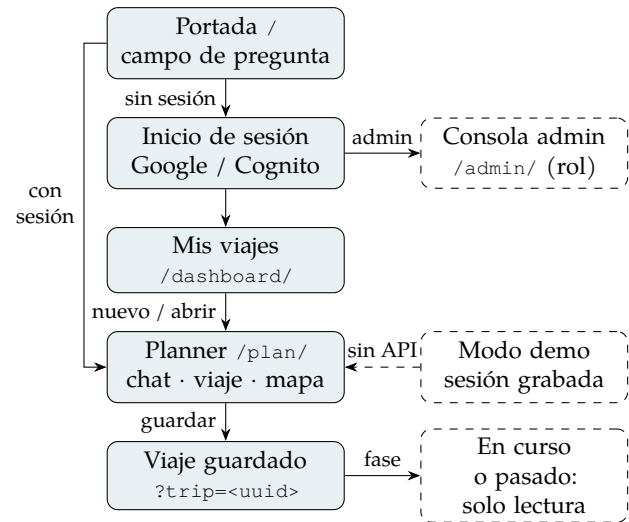


Figura 1: Recorrido del usuario. En trazo discontinuo, superficies laterales al flujo principal.

Ruta	Función
/	Portada: un único campo de pregunta
/auth/callback/	Retorno del login gestionado de Cognito
/dashboard/	Mis viajes: campo de pregunta y viajes por fase
/plan/	Planner (?q= o ?trip=): chat, viaje y mapa
/trip/?id=	Redirección heredada a /plan/?trip=
/admin/	Consola: KPIs por día, modelo y ciudad
/admin/...	turns/+turn/ (inspector), trips/+trip/, users/ (rol y subject)
otra	404 exportado como 404.html

Tabla 2: Páginas del export estático. Los identificadores van en la *query string* porque un export estático no conoce en compilación los ids de los usuarios (ADR 0011).

la ciudad (ADR 0019). A diferencia de un chat genérico, cada recomendación es una tarjeta anclada en un documento del corpus (Wikivoyage, Wikipedia, OpenStreetMap), con foto y atribución (sección 6).

En el recorrido (figura 1 y tabla 2), el login (Google en local, Cognito con PKCE en producción) conserva el destino `/plan/?q= y ?trip=<uuid>` reconstruye el borrador. Sin `ai_api`, un *modo demo* avisado reproduce una sesión de Budapest; la consola `admin` es de solo lectura (ADR 0024).

2.1 El planner

En escritorio, tres columnas (chat $\approx 30\%$, viaje $\approx 40\%$, mapa $\approx 30\%$); en móvil, pestañas. Un *brief* (ciudad, fechas, origen, presupuesto, intereses), completado con preguntas cortas y chips, da paso a carruseles de tarjetas: barrios, hoteles por nivel de precio (€ a €€€; nunca un número) y actividades por franja (mañana, tarde, atardecer, noche), con foto acreditada, enlace a la fuente y “Cambiar” para ver alternativas. Los lugares citados en prosa llegan “sin colocar” y el viajero elige día y franja: el sistema nunca mueve su mañana sin pedirlo (ADR 0018).

El panel central muestra la ruta (enlace a Google Flights, sin precio), la estancia, una tira de días con previsión (Open-Meteo [1] a corto plazo, normales climáticas del corpus más allá) y una vista general de la ciudad. El mapa (MapLibre GL [2], teselas de OpenFreeMap [3]) numera las paradas del día unidas por líneas rectas, sin cálculo de rutas.

Guardar crea un `Trip` en `core_api`. *Contexto:* el modelo previo era multidestino con estado almacenado. *Decisión* (ADR 0019): una ciudad por viaje y fase (próximo, en curso, pasado) derivada de las fechas en cada respuesta; un viaje en curso o pasado es de solo lectura (409 `TRIP_LOCKED`, sin campo de texto). *Consecuencia:* la fase no puede desincronizarse, pero la migración borró los viajes multidestino y la fase se calcula en UTC. *Valoración:* recorte legítimo para un producto mínimo, pero excluye interrail y rutas multicidadad. ADR 0020 creó “Mis viajes” días después porque el login dejaba un planner vacío: la entrada no se validó antes.

2.2 Identidad, móvil y accesibilidad

La identidad (`docs/design/kyrian-world.md`) parte de que “el campo es lo memorable”: sin rejilla de funciones ni testimonios, solo un fondo de “aurora” (`aria-hidden`), tema oscuro, variables CSS, Outfit y Plus Jakarta Sans. La consola, a propósito, es densa y usa gráficos SVG hechos a mano.

Inglés y español cambian sin recarga; tipos y un test exigen las mismas claves. En móvil (TRA-187), 100dvh evita que la barra del navegador tape el campo, cada panel desplaza por separado y los controles miden 16px en punteros táctiles (sin zoom de iOS), comprobado por E2E a 390×844. Según WCAG 2.2 [4]: contraste $\geq 4,5:1$ calculado token a token (el dorado del tema claro bajó a #8A5400, 5,8:1, tras descartar dos tonos), un único contrato de diálogo (foco atrapado, Escape lo devuelve), `prefers-reduced-motion` global y tablas `sr-only` tras los gráficos de la consola. Sin auditoría automatizada (axe, Lighthouse) en CI, el cumplimiento está declarado, no medido.

2.3 Fotos: mejor ninguna que la equivocada

Contexto: tiene foto el 80–84 % de los documentos de monumentos y visitas, pero solo el 0–2 % de restaurantes y el 6–12 % de hoteles, y el recurso previo mostraba la foto de *otro* local. Google Places Photos y TripAdvisor se descartaron por sus términos. *Decisión* (ADR 0021): cascada imagen del corpus → Wikimedia Commons → `og:image` del sitio del propio local (no persistida, con listas de bloqueo) → foto de un monumento solo para barrios → marcador neutro. Para hoteles, ADR 0022 resuelve la foto al construir el corpus (sitio propio, Facebook, Commons por nombre, mayor imagen de la portada) y *un hotel sin foto sale del corpus*: con foto localizada, Budapest pasó de 36 a 206 de 415, Berlín 338/649 y Madrid 279/527 (antes del filtrado; la tabla 11 da los que quedan tras él). *Valoración:* confianza antes que inventario, a costa de casi la mitad de los hoteles, de enlaces directos que pueden romperse hasta la siguiente reconstrucción y de una capa curada a mano (cadenas) sin verificación automática.

Stack. Export estático de Next.js 16 [5] en S3 y CloudFront (sección 7). El lint impone que `fetch` y los tipos generados desde OpenAPI solo vivan en `src/services/`, donde un mapeador los convierte en modelos de vista (ADR 0006); el estado del planner es un reductor puro probado sin React (tabla 3).

Elemento	Valor
Framework	Next.js 16.3, React 19.3, TypeScript 5.9 estricto
Estilos, mapa	Tailwind v4 sin <code>tailwind.config.js</code> ; MapLibre GL 6.10
Pruebas	Vitest 5, Testing Library, Playwright 1.63 (3 configuraciones)
<code>.ts/.tsx</code>	259 ficheros (83 de test); 34 623 líneas (22 333 sin tests)
Tipos generados	2 ficheros aparte, 4 664 líneas (<code>core-api.ts</code> , <code>ai-api.ts</code>)
Tests unitarios	747 casos en 83 ficheros, todos pasan (Vitest, 23-09-2026)
Specs E2E	9 ficheros, 1 917 líneas (no ejecutados para esta memoria)

Tabla 3: Stack y tamaño del frontend (`package.json`, recuento y ejecución en `src/frontend`).

Valoración. El producto resuelve un caso acotado (una ciudad, un viajero, hasta una semana). Las tarjetas no se reordenan, no hay horarios ni notas, y guardar reescribe una instantánea completa (TRA-146). No hay colaboración, reservas, precios reales, PDF ni aplicación instalable. El mapa depende de un servicio gratuito sin SLA y el emparejamiento de lugares en la prosa es heurístico. Faltan mejoras obvias: enseñar en la tarjeta el fragmento recuperado, que es el diferenciador (hoy solo el enlace a la fuente, con la licencia en un *tooltip*); tarjetas en el idioma del usuario (hoy en inglés); tiempos de desplazamiento entre paradas, y una valoración por tarjeta ligada a las trazas (sección 12).

3 Estudio de mercado y viabilidad

3.1 Mercado y tendencias

El turismo internacional cerró 2025 con 1 520 millones de llegadas (+4 %); Europa recibió 793 millones [6]. España batió su récord con 96,8 millones de turistas internacionales y 134 712 M€ de gasto [7]. Las OTA facturaron 94 000 M\$ en 2024 (+7 % anual) y ven en la IA conversacional una amenaza [8]. Los ingresos se generan al *reservar*; *planificar*, el eslabón que cubre Kyrian World, apenas se monetiza por sí mismo.

En España el 40 % de los adultos ya ha usado un asistente de IA para preparar un viaje (el 73 % entre los 18 y los 24 años) [9]; Europa va por detrás (Reino Unido 22 %, Francia 19 %, Alemania 15 %) [10]. Falta confianza: solo el 17–20 % de los viajeros europeos confía plenamente en una IA [11], y el 49 % de los españoles que no la usan teme recomendaciones inexactas o desactualizadas [9]. En una encuesta patrocinada por Mindtrip a 103 organizaciones de gestión de destinos (DMO) de EE. UU., solo el 22 % ofrece un planificador con IA en su web [12].

3.2 Competencia

La tabla 4 enseña que los generadores sin datos propios acaban comprados y que Expedia retiró Romie por responder sin inventario ni precios reales [13].

Posicionamiento. Kyrian World compite en *verificación*, no en distribución ni inventario: cada tarjeta remite, con enlace a la fuente, a un documento de un corpus con licencia limpia (sección 6). Es un anclaje documental (existencia, descripción, fuente), no de disponibilidad ni precio: cubrir la carencia de Romie exige in-

tegrar inventario (horizonte de 12 meses). Asume una ciudad por viaje (ADR 0019) y pocas ciudades, aunque en castellano. Su ventaja es el coste fijo: ≈ 5 €/mes en recursos de Terraform (ADR 0023) más ≈ 8 €/mes de un WAF creado a mano, unos 12–13 €/mes (estimación; la factura de septiembre debe sustituirla), frente a rondas de millones.

3.3 Monetización

La **afiliación** paga por reserva completada (tabla 5), pero el modelo por sesión de Booking.com solo computa la reserva hecha en el navegador inicial [20]. La **suscripción** toma como referencia Layla Premium, a 9,99 \$/mes o 49,99 \$/año [14]; se supone un plan de 39 €/año con una conversión del 1–2 % (*hipótesis*: el contenido del plan *premium* está sin definir). La vía con más recorrido es el **B2B o marca blanca para DMO**, con demanda demostrada por GuideGeek [17] y Mindtrip [16]: se vendería el *pipeline* `/add-city`, a 6 000–15 000 € por destino y año (*hipótesis*).

3.4 Economía unitaria

Con unas 12 llamadas al modelo por viaje, $\approx 50 000$ *tokens* de entrada y $\approx 8 000$ de salida (*estimación*), el coste variable por viaje es de 0,09 \$ con Claude Haiku 4.5, el modelo de producción; 0,18–0,30 \$ con Sonnet (el *endpoint* de la UE es un 10 % más caro) [25]; 0,07 \$ con Nova Pro [26]; vectores y *embeddings*, $<0,001$ \$. El escenario usa 0,20 €/viaje para dejar margen a reintentos, llamadas de chat y un posible modelo mayor.

El ingreso por afiliación es de 0,37 € por viaje en el escenario base (0,15–0,76 €), con tasas de clic y conversión *supuestas*: el margen queda en $\approx 0,17$ € por viaje, y en el escenario pesimista cada viaje pierde dinero. En B2C, el LTV ronda 1,8 € (*hipótesis*: 1,8 viajes/año $\times$ 0,17 € más 1,5 % $\times$ 39 € de *premium*, $\approx 0,9$ €/año durante dos años). Como CAC se usa de *proxy* el coste por instalación de una app móvil de viajes, 4,20 \$ [27]: LTV/CAC $\approx 0,5$, así que el B2C depende de canales orgánicos. En B2B, un contrato de 8 000 €/año durante tres años frente a 2 000–4 000 € de coste de venta pública da un LTV/CAC de 6–12 (*hipótesis*).

En la tabla 6 el año 1 cuadra solo porque no imputa salario: con un salario mínimo imputado daría $\approx -15 000$ € (*hipótesis*). Cubrir 35 000 € de salario y 5 000 € de fijos exigiría, solo con afiliación, unos 235 000 viajes al año, irreal sin adquisición de pago; con B2B bastan cinco contratos de $\approx 8 000$ €.

3.5 Marco legal

Entre corchetes, el estado de cada obligación. El **RGPD** exige base jurídica, minimización, plazos de conservación de las trazas y cuidado con el texto libre, que puede revelar categorías especiales [25]; [*pendiente*: el login menciona una política de privacidad y unos términos cuyas páginas no existen]. El **Reglamento de IA (UE) 2024/1689** no incluye un planificador de viajes en el Anexo III: le afecta el art. 50 (desde el 02-08-2026): avisar de que se habla con una IA y marcar de forma legible por máquina el contenido sintético [28] [*parcial*: el planner se titula “Dile a la IA adónde quieres ir”, sin aviso explícito ni marcado]. La **Directiva de viajes**

Nombre	Tipo / propietario	Negocio	Diferenciador	Debilidad o estado
Layla	<i>Startup</i> (≈5 M€)	<i>Freemium</i> (9,99 \$/mes), afiliación [14]	Chat y vídeo; precios en vivo	Comprada por Expedia (07-2026); mantiene su marca [15]
Roam Around	<i>Startup</i> (1 M\$)	Gratuito	Sencillez; auge tras ChatGPT	Absorbida por Layla (02-2024); sin datos propios
Mindtrip	<i>Startup</i> (19 M\$)	Gratis; comisión; B2B	11 M de POI; >35 destinos clientes [16]	Fuerte en EE. UU.; poco español
GuideGeek	Matador Network	Marca blanca para DMO	>70 DMO; WhatsApp, Instagram [17]	Depende de OpenAI; generalista
Wonderplan	<i>Startup</i>	Gratis, sin registro	Rapidez; PDF	Casi insensible al perfil
Wanderlog	<i>Startup</i> (YC)	<i>Freemium</i>	Colaboración y mapas	IA secundaria
ChatGPT	OpenAI	Suscripción; apps de Booking y Expedia (10-2025) [18]	Distribución masiva	Sin anclaje; reserva agéntica experimental
Google AI Mo- de	Alphabet	Publicidad, comisión	Itinerarios (11-2025); datos de Maps [19]	Solo EE. UU.; Google Trips cerró en 2019
Expedia Romie	Expedia Group	Comisión OTA	Planificación y soporte (2024)	Retirado: sin inventario ni precios reales [13]
Booking.com	Booking Holdings	Comisión OTA	Chat sobre inventario propio (2023)	Ligado a su inventario; app en ChatGPT

Tabla 4: Competidores y comparables (septiembre de 2026). Las cifras sin cita proceden del estudio de mercado del proyecto (reseñas y prensa sectorial).

Programa	Comisión de referencia
Booking.com	4 % de estancias completadas; por sesión [20]
Expedia Group	Hoteles 3–4 %; actividades 4 % [21]
GetYourGuide / Viator	≈8 %; <i>cookie</i> de 30 días [22]
Kiwi.com	3 % del billete [23]
Omio	2–8 % según mercado y volumen [24]

Tabla 5: Comisiones de afiliación de referencia (consultadas el 23-09-2026).

Concepto (€)	Año 1	Año 2	Año 3
Viajes planificados	5 200	40 000	162 000
Afiliación (0,37 €/viaje)	1 924	14 800	59 940
<i>Premium</i> (1–2 % × 39 €)	1 560	14 625	70 200
B2B (contratos)	3 000 (1)	32 000 (4)	108 000 (12)
Ingresos	6 484	61 425	238 140
peso del B2B	46 %	52 %	45 %
Coste variable (0,20 €/viaje)	1 040	8 000	32 400
Infra., datos y legal	1 980	7 440	19 800
Marketing y venta B2B	3 000	28 000	95 000
Resultado antes de salarios	+464	+17 985	+90 940
Salarios	0	35 000	90 000
Resultado neto	+464	–17 015	+940

Tabla 6: Escenario base a tres años (10-2026 a 09-2029). Hipótesis: 4 000 / 25 000 / 90 000 usuarios activos con 1,3 / 1,6 / 1,8 viajes al año; B2B a 3 000 € (piloto), 8 000 € y 9 000 € por contrato; fundador asalariado desde el año 2 y una segunda persona en el año 3.

combinados no aplica mientras Kyrian World solo redirija. Las **licencias** imponen atribución: CC BY-SA 4.0 (Wikipedia, Wikivoyage) obliga a compartir igual las adaptaciones publicadas, como el corpus JSONL [29]; ODbL exige atribuir a OpenStreetMap [30] [*parcial*: la tarjeta enlaza la fuente, pero la licencia solo aparece en un *tooltip*; la introducción de la ciudad sí cita “Wikivoyage (CC BY-SA 4.0)”]. La API gratuita de **Open-Meteo** es solo no comercial [31] [*pendiente*: monetizar exige su plan comercial u otra fuente].

Vender a una DMO pública española. La licencia hipotética de 6 000–15 000 € encaja en un contrato menor de servicios (art. 118 LCSP, <15 000 €) [32]. Exigiría además conformidad con el Esquema Nacional

Fortalezas	Debilidades
– Anclaje citado, licencia limpia – 12–13 €/mes de coste fijo – Alta de ciudad reproducible	– Pocas ciudades; sin reservas – Sin métricas ni ingresos – <i>Bus factor</i> 1
Oportunidades	Amenazas
– 78 % de DMO sin IA (EE. UU.) – Red DTI: 440 destinos – Art. 50; LLM más barato	– Google y ChatGPT gratis – Expedia compra Layla – Comisiones y datos abiertos

Tabla 7: DAFO de Kyrian World (septiembre de 2026).

de Seguridad [33], sin evaluar; accesibilidad según el RD 1112/2018 y la EN 301 549 [34, 35], declarada pero no auditada (sección 2); alojamiento en la UE, que la región `eu-west-1` cubre para la API pero sin acreditación formal; contenido en español, cuando el 98,8 % del corpus está en inglés, y cobertura de ciudades medianas, donde Bolonia tiene solo 1 597 documentos. Hoy ninguno está acreditado.

3.6 Conclusión de viabilidad

Como B2C financiado con afiliación (tabla 7), la viabilidad es marginal: el ingreso por viaje es del orden del coste del modelo, el LTV no cubre el CAC y las plataformas regalan la planificación. El nicho recomendado es el B2B para oficinas de turismo de ciudades europeas medianas y los 440 destinos de la Red de Destinos Turísticos Inteligentes [36], con el viajero hispanohablante como segmento secundario. El diferenciador a defender es la *IA verificable con fuentes*, en línea con la transparencia del art. 50; la carencia de Romie [13], inventario y precios reales, sigue sin cubrir.

4 Equipo y contribución individual

El equipo lo forman cuatro personas. La tabla 8 resume, según *git* y *GitHub* (PR fusionadas en `main` hasta `e7916c8`), qué hizo cada una; el rol se deduce de las rutast del repositorio que tocó y de los títulos de sus PR, no de un reparto formal previo.

Tabla 8: Contribución por miembro según las PR fusionadas (181 en total, 3 de ellas de Dependabot).

Miembro (PR)	Rol principal, áreas del repositorio e hitos (n.º de PR)
Manuel Pérez (145)	Producto de extremo a extremo y proceso de entrega con agentes. <code>src/frontend</code> , <code>core_api</code> , planificador de <code>ai_api</code> , <code>city_corpus</code> , migración a DynamoDB, consola de administración (#193–#196), <code>infra/aws</code> v3, CI/CD, docs y ADR. Por prefijo del título: ≈27 infraestructura/CI, 22 <code>frontend</code> , 21 <code>core_api</code> , 20 documentación, 17 corpus, 17 <code>ai_api</code> , 8 administración, 13 otras.
Alexander De Sousa (19)	Prototipo del planificador (#17–#20); <i>scraper</i> de Google Places y Wikipedia (#26–#31, #68); <i>devcontainer</i> inicial (#35, #36); Terraform inicial de AWS y GCP (#37); <i>frontend</i> HTTPS con S3, CloudFront, Route 53 y ACM (#83); corpus de Madrid (#172).
Joaquín Castro Salas (14)	Andamiaje del <i>backend</i> y esquema de datos (#21–#25); OAuth de Google, chat, CI/CD (#32); protección del chat y CI del <i>backend</i> (#40, #41); Bedrock en <code>ai_api</code> (#109); hilos de chat (#111, #115); infraestructura de S3 Vectors (#116); RAG del chat con S3 Vectors y Titan (#124); corpus de Berlín (#171).
David Baos (0; 1 rama)	Ciudad de Miami a través del <i>pipeline</i> de <code>city_corpus</code> (TRA-204, rama <code>tra-204-miami-through-the-pipeline</code> , 21-09-2026): configuración <code>cities/miami.toml</code> , corpus de 1249 documentos, manifiesto e informe de <i>readiness</i> ; sin PR fusionada en <code>main</code> en la fecha de corte.

Cómo se registró la contribución. Todo cambio sigue la misma cadena: issue en Linear (TRA-*n*) → rama `<tipo>/TRA-n-<slug>` → PR con autor en GitHub → fusión. El código generado con agentes lleva en el *commit* la línea `Co-Authored-By: Claude (151 de los 324 commits de main)`; aun así, cada PR la abrió, revisó y fusionó una persona, que es su autora a efectos de esta tabla (sección 9).

Limitaciones de la evidencia. El reparto es desigual (145/19/14/0 PR fusionadas) y el recuento de PR o de líneas no mide esfuerzo: no captura el trabajo no versionado (reuniones, curación manual de datos, pruebas en producción) ni el peso de cada PR, y deja fuera el trabajo que aún no ha llegado a `main`, como la ciudad de Miami.

5 Arquitectura de software

Un frontend Next.js exportado como sitio estático y dos servicios FastAPI [37] que solo comparten la forma de verificar el token (figura 2). El pipeline de IA se trata en sección 6 y el despliegue en sección 7.

5.1 Dos servicios, una frontera y un núcleo compartido

ADR 0001 dividió el FastAPI inicial (unas 3 150 líneas) porque el RAG traía dependencias pesadas, *streaming* largo y otro perfil de escalado, y el único acoplamiento era `get_current_user` leyendo la base de datos. La frontera se verifica: `ai_api` y `core_api` no se importan entre sí, y `tests/test_import_boundaries.py` comprueba además que solo `infrastructure/dynamo/` importa `boto3`. Lo compartido vive en `travel_common`:

`Principal`, ajustes, errores de dominio, verificación de tokens, *app factory* y *fakes*. Todo forma un *workspace* uv con un único `uv.lock`, `ruff` con reglas ampliadas (E4/E7/E9/F, I, UP, B, SIM, N, RUF, ASYNC, S; el *scraper* conserva E/F) y *pyright standard*.

5.2 Capas frente a puertos y adaptadores

Un CRUD usa capas; un servicio de integración, puertos [38]. `core_api` sigue `api` → `services` → `domain`, con `infrastructure/dynamo/` como único adaptador y casos de uso sin FastAPI ni `boto3`. `ai_api` define en `domain/ports.py` protocolos (`LLMProvider`, `Embedder`, `Retriever`, `TraceLog`, ...) que implementa `infrastructure/`; cambiar de proveedor LLM toca solo esa capa y `api/deps.py`.

`Trip` es el agregado raíz (ADR 0005): embebe días (con actividades y comidas), alojamientos y transportes, y se lee y guarda entero. Los hijos cuelgan de `/trips/{trip_id}/...`, autorizados una vez, y `child_router()` genera 25 de las 49 rutas; así se cerró una brecha real (se podían leer hijos ajenos) y desaparecieron unas 600 líneas copiadas. ADR 0019 fijó un viaje por ciudad y una fase (*upcoming*, *ongoing*, *past*) derivada de las fechas, nunca almacenada; `ensure_editable()` rechaza con 409 `TRIP_LOCKED` las escrituras en viajes en curso o pasados, y tras cada `PATCH` se reejecutan las invariantes de la entidad. Perfil, viaje y conversación llevan `version`: las escrituras son condicionales y una carrera perdida es 409 `CONFLICT` (bloqueo optimista). Los servicios lanzan solo errores de dominio, que un único manejador traduce a `{message, error_code, extras}`; ningún *endpoint* usa `HTTPException`. La identidad es un `Principal` (`subject`, `email`, `role`), ampliado en `core_api` con el UUID de la cuenta.

5.3 Identidad y confianza entre servicios

`AUTH_MODE` elige el emisor: en `local`, `core_api` verifica el token de Google con `tokeninfo` y emite un JWT HS256 [39] propio; en `cognito` (desplegado), un *user pool* con Google como IdP emite RS256 verificado *offline* contra el JWKS [40]. ADR 0009 cambió el emisor de ADR 0002 (JWT con secreto compartido) pero mantuvo la regla: `ai_api` reenvía a `core_api` el *bearer* del usuario, con sus mismos permisos y sin secreto entre servicios. `core_api` comprueba la cuenta en cada petición y corta al instante a un usuario desactivado; `ai_api` no, y ese usuario sigue usando la IA hasta que expira el token (60 min), riesgo aceptado en ADR 0002.

5.4 Tabla única en DynamoDB

ADR 0023 retiró el RDS PostgreSQL de ADR 0009: costaba unos 15 €/mes, la mitad del presupuesto de 30 €/mes; era la única razón de la VPC; y los accesos eran de clave-valor, sin `join`, `group_by` ni `ilike`, y el viaje siempre entero. La tabla única [41] (tabla 9) resuelve cada patrón con una *Query* y cuesta menos de 1 €/mes estimado. Costes reconocidos en el ADR: la integridad pasa a la aplicación (ítem `EMAIL#` en la misma transacción, borrados en cascada a mano), no hay migraciones (un campo ausente toma el valor por defecto de la *dataclass*), las preguntas *ad hoc* exigen código

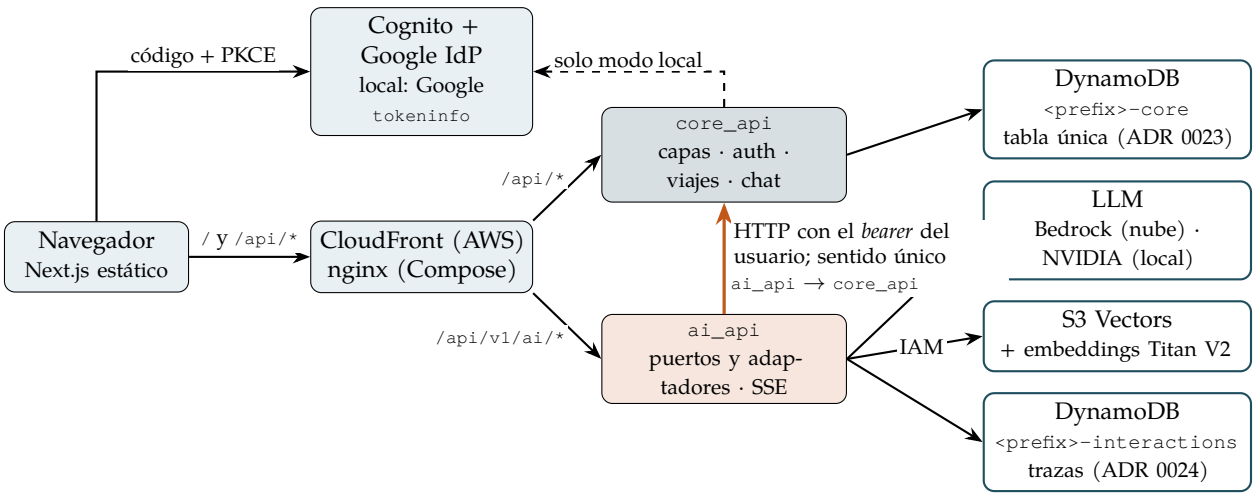


Figura 2: Contenedores y flujos. `core_api` funciona con `ai_api` caído; ambos verifican el JWT por su cuenta (HS256 local o RS256 con el JWKS de Cognito) y cada almacén tiene un único servicio propietario.

Tabla 9: Ítems de la tabla `<prefix>-core` (ADR 0023).

Ítem	PK	SK	Índice / nota
Cuenta	USER#<id>	PROFILE	GS1 USERS/email
Email	EMAIL#<email>	EMAIL	unicidad, misma transacción
Viaje	USER#<id>	TRIP#<id>	agregado entero; GS12 TRIPS (admin)
Hilo	USER#<id>	THREAD#<id>	sin sus mensajes
Mensaje	THREAD#<id>	MSG#<ts>#<id>	append-only, orden por μ s

Patrones: perfil (PROFILE); viajes o hilos (begins_with(SK)); hilo en orden (MSG#); administración por GS1/GS12.

o exportación, y “otro usuario” pasa de 403 a 404 porque la clave incluye al propietario. `ai_api` escribe en `<prefix>-interactions` (ADR 0024): cada turno es una traza con resumen (DAY#), contexto y pasos (TURN#), dos GSI (por usuario y por sesión del planificador) y TTL de 90 días (sección 6). La traza guarda el mensaje del usuario, el historial (hasta 8 KB) y la respuesta durante esos 90 días, visibles para el grupo `admin`: un dato personal en texto libre (sección 12).

5.5 Contratos tipados

Pydantic [42] es la fuente de verdad: `just contracts` exporta `docs/api/*.openapi.json` y regenera los tipos TypeScript; CI falla ante cualquier diferencia. El planificador emite eventos SSE tipados (ADR 0015): una unión discriminada en `type` (`text`, `brief`, `options`, `itinerary_patch`, `error`, `done`) cuyas operaciones de itinerario son otra unión en `op`. Ningún campo es opcional en el cable (lo desconocido viaja como `null`), así que el TypeScript no tiene `?`. Sin `response_model` en el `streaming`, `ai_api/openapi.py` inyecta los modelos. El orquestador es sin estado: cada petición lleva `brief`, `itinerary` y `transcripción`, y los `ids slot:<day>:<part>` se interpretan sin sesión.

Las pruebas (tabla 10; los casos ejecutados superan a las funciones por la parametrización) no necesitan infraestructura (moto en proceso, proveedores falsos, emisor Cognito de prueba); los PR que tocan `prompts` exigen además una prueba manual con el modelo real.

Tabla 10: Tamaño y pruebas por paquete: líneas contadas en el commit `e7916c8`; tests ejecutados con `just test` el 23-09-2026, todos en verde.

Paquete	LOC prod.	LOC tests	Tests	Rutas
travel_common	829	414	37	—
core_api	3 709	3 111	199	49
ai_api	10 691	9 119	597	10
Total	15 229	12 644	833	59

5.6 Valoración

Hay deuda. El doble modo de autenticación es ya permanente: duplica pruebas y configuración, y `AUTH_MODE`, `SECRET_KEY` y `COGNITO_*` deben coincidir a mano en dos `.env` sin test que lo compruebe. La revocación de 60 min en `ai_api` y la falta de límites de uso exponen seguridad y coste. En producción solo hay un proveedor LLM, Bedrock, sin conmutación, aunque el puerto `LLMProvider` lo permitiría. La documentación está desfasada: `docs/architecture/code-quality-review.md` describe la arquitectura SQLAlchemy anterior a ADR 0023 sin equivalente para DynamoDB, ADR 0013 no se marcó como sustituido en su almacenamiento y quedan directorios vacíos (`db/`, `models/`, `repositories/`, `seed/`) de la capa anterior. Sección 12 prioriza las propuestas.

6 Arquitectura de la IA

El sistema es un RAG [43] con una restricción que lo condiciona todo: *toda tarjeta procede de un documento recuperado del corpus* y los ids no recuperados se descartan. Lugares, fotos, horarios y precios salen de un corpus por ciudad construido fuera de línea desde fuentes abiertas y comprometido en git; el LLM interpreta la petición, elige entre ids recuperados y redacta frases cortas. Esa prosa libre (respuestas de chat, el `why` de 140 caracteres) sí puede mencionar lugares y no está verificada. El turno (figura 3) corre en la Lambda de `ai_api` (512 MB, 900 s, streaming; sección 7).

6.1 El corpus por ciudad

Fuentes y licencias. Solo entran licencias compatibles (no Google Places, TripAdvisor ni Booking): Wiki-

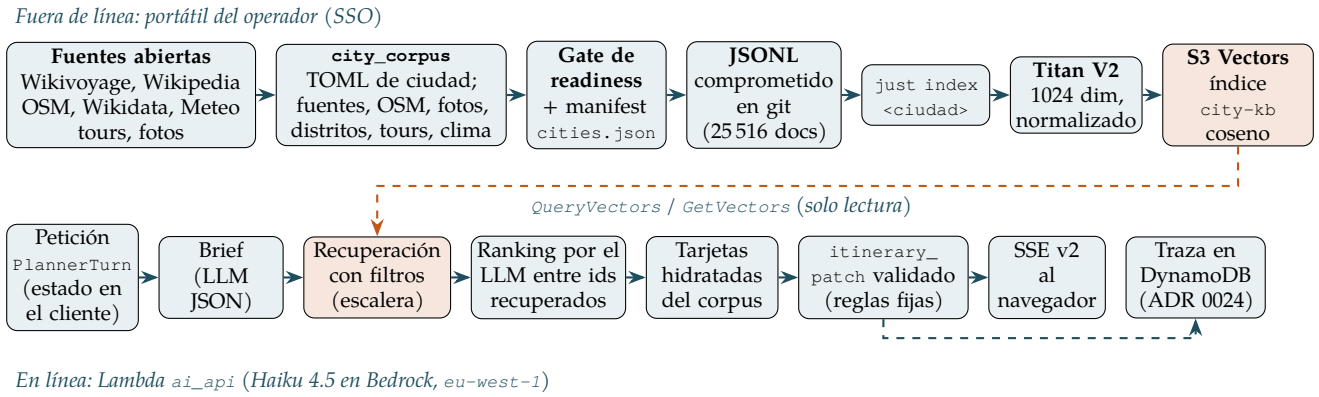


Figura 3: Pipeline de IA. Arriba, construcción reproducible: el JSONL comprometido es la fuente de verdad y el índice se regenera con un comando. Abajo, un turno del planner.

Tabla 11: Corpus actual (conteo propio de `documents.jsonl`; fecha de construcción, 2026). Hoteles: `sleep` con foto / total tras el filtrado de ADR 0022; sección 2 da los localizados antes de filtrar.

Ciudad	Docs	en / es	Hoteles	Fecha
Berlín	12 240	12 187 / 53	341/378	22-09
Budapest	6 105	5 993 / 112	212/229	22-09
Madrid	5 574	5 501 / 73	267/272	22-09
Bolonia	1 597	1 532 / 59	3/118	19-09
Total	25 516	98,8 % en		

voyage en/es y Wikipedia, CC BY-SA 4.0 [29]; OpenStreetMap (ODbL [30]) para hoteles, restaurantes, bares y distritos; Wikidata (CC0) y Commons (sin ficheros NC, ND ni de uso legítimo) para enriquecer, y Open-Meteo (CC BY 4.0) para doce normales. Si `source`, `source_url` y `license` no casan, la construcción falla. Única excepción: fotos de la web del propio hotel, con el dominio como crédito (ADR 0022). Cada línea de `documents.jsonl` lleva `doc_id` estable, distrito, `category`, `kind` (`listing/prose`), texto, coordenadas, horario, `price_tier` y atribución. Los `listings` son atómicos; la prosa se trocea por encabezado en ≤ 500 tokens con un párrafo de solape.

Ciudades, gate y manifest. Una ciudad es un `cities/<slug>.toml` (borrador de `just corpus-discover`) más su corpus (tabla 11); el planner no conoce ninguna por su nombre. El *gate* de *readiness* define un corpus terminado: ≥ 150 lugares de visita y ≥ 100 de comida localizados, ≥ 10 hoteles con foto, ≥ 5 distritos con guía de barrio, 50 % de monumentos con foto, 12 normales y ≥ 3 tours. Los tours son la única entrada manual (`curated/<slug>/tours.toml`), redactada desde la web del operador, nunca de agregadores. El manifest `cities.json` (ADR 0017) publica alias, centro, introducción y portada: añadir una ciudad no toca Terraform. Los hoteles sin foto, cerca de la mitad (sobre todo por webs caídas), salen del corpus (ADR 0022). Bolonia (3/118) se construyó antes: hoy no superaría el gate de 10 hoteles con foto y sigue publicada (los filtra el planner), riesgo aceptado y pendiente de reconstrucción. Cada tarjeta toma la imagen del corpus, de Commons cercano, la `og:image` del local o un marcador (ADR 0021). El tope de 2 000 documentos por ciudad (TRA-210) **no**

está implementado: Berlín lo supera 6,1 veces, con un 53 % de restaurantes de OSM sin descripción.

6.2 Embeddings y almacén vectorial

Documentos y consultas se embeben con Titan Text Embeddings V2 [44] (1024 dimensiones, normalizado, sin asimetría consulta/documento), multilingüe: una pregunta en español encuentra prosa inglesa. El almacén es Amazon S3 Vectors [45] (ADR 0014): un índice `city-kb` (`float32`, 1024, coseno) creado por Terraform y compartido, con `city` como metadato filtrable y clave `uuid5(doc_id)` porque 1 357 ids llevan acentos. Los metadatos filtrables (`category`, `district`, `price_tier`, `lat`, `lon`...) ocupan como mucho 185 B de 2 KB. Se acepta búsqueda **solo densa** y **sin filtro geográfico** (un radio son cuatro comparaciones sobre `lat/lon`); la fusión BM25 [46] + RRF [47] ($k = 60$) solo existe en el banco del spike.

6.3 El spike TRA-151: Qdrant en Lambda frente a S3 Vectors

ADR 0014 eligió S3 Vectors por simplicidad operativa, antes de medir; el spike (`docs/architecture/vector-store-spike.md`, 17-09-2026) fue confirmatorio. Comparó las dos opciones viables tras descartar `pgvector` en RDS (inalcanzable desde una Lambda sin VPC), Qdrant Cloud gratuito (se suspende tras una semana inactivo) y Qdrant en Fargate (16 €/mes expuesto, 30 €/mes privado: el doble, y ya el presupuesto íntegro de ADR 0009). A es el binario de Qdrant [48] tras Lambda Web Adapter, con la colección horneada en la imagen y desempaquetada en `/tmp` en cada arranque en frío; B, S3 Vectors.

Método. Ambos se llenaron con los mismos vectores de los 6 082 documentos de Budapest de entonces, con el coseno exhaustivo como referencia, 32 consultas y 52 `doc_id` esperados (4 en español sobre documentos en inglés). La latencia se midió desde una Lambda sin VPC en `eu-west-1`, como `ai_api`: 128 consultas en caliente por candidato y memoria y arranques en frío forzados.

Resultados. En calidad (tabla 12) no hay diferencia apreciable: un documento en una consulta, con 32 con-

Tabla 12: Calidad medida (32 consultas). Agr.: fracción del top-10 exhaustivo que devuelve el almacén.

Estrategia	R@5	R@10	MRR	Agr.
Coseno exhaustivo	0,688	0,812	0,591	—
B: S3 Vectors	0,688	0,812	0,591	0,969
A: Qdrant en Lambda	0,672	0,797	0,591	0,963
Solo BM25	0,578	0,646	0,505	0,353
Exhaustivo + BM25 (RRF)	0,693	0,760	0,651	0,644

Tabla 13: Latencia medida (ms). *search*: el almacén; *total*: con Titan, de p50 72–75 ms en todos los grupos.

Cand.	MB	Caliente			Frío	
		search p50	p95	total p50	search	total
A	2048	21,5	27,1	93,5	1959	2054
A	1024	22,5	29,3	97,7	1717	1824
B	2048	64,2	69,1	136,9	91	189
B	1024	66,5	152,3	140,1	288	387

sultas de una sola ciudad (4 en español) y sin intervalos de confianza. La fusión híbrida sube el MRR de 0,591 a 0,651 y baja R@10 de 0,812 a 0,760. En caliente (tabla 13) Qdrant es tres veces más rápido, pero el embedding domina y todo es despreciable frente a un LLM; en frío, el caso habitual con tráfico de demostración, el orden se invierte. El coste (tabla 14) es ruido frente a los 30 €/mes del presupuesto.

Decisión. Se mantuvo S3 Vectors porque *deciden las operaciones* y el arranque en frío: B es un índice y un comando (6082 vectores en 14 s); A, una imagen de 125 MB y una colección que se reconstruye con cada cambio del corpus y que tres veces se publicó vacía informando de éxito. Las tarjetas, que son el producto, se sirven con búsqueda densa sola: no activar BM25 + RRF es una deuda asumida sobre lo medido, a cambio de no mantener un índice BM25 en proceso por ciudad y de no perder R@10.

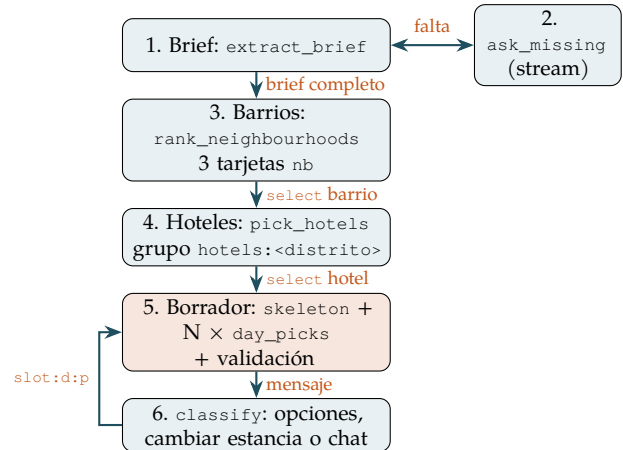
6.4 Modelos preentrenados: por qué no hay entrenamiento ni ajuste fino

No se entrena ni se ajusta ningún modelo: se consumen modelos fundacionales preentrenados por API. En producción, Claude Haiku 4.5 de Anthropic en Amazon Bedrock (perfil europeo eu.anthropic.claude-haiku-4-5-20251001-v1:0, API Converse [49], 4096 tokens de salida) hace todo el trabajo generativo, y Titan Text Embeddings V2 (amazon.titan-embed-text-v2:0), los embeddings. En local, el mismo puerto habla con nvidia/nemotron-3-super-120b-a12b por la API alojada de NVIDIA, compatible con OpenAI. eu.amazon.nova-lite-v1:0 está configurado para títulos, pero ningún código lo usa.

Se prefirió RAG con ingeniería de *prompts* al ajuste

Tabla 14: Coste mensual estimado por volumen (precios de lista de us-east-1, no de eu-west-1; Titan añade a ambos ~0,002 USD por 100 000 consultas).

Consultas/mes	A: Qdrant Lambda	B: S3 Vectors
1 000	~0,07 USD	~0,005 USD
10 000	~0,08 USD	~0,03 USD
100 000	~0,09 USD	~0,25 USD

**Figura 4:** Fases de PlanTrip. Sin estado en servidor: la fase se deduce del brief, del *snapshot* y del *group_id* de la acción.

fino porque el conocimiento que importa es factual, local y cambiante (hoteles, horarios, fotos, tours): se actualiza reconstruyendo un JSONL y reindexando por céntimos, y solo la recuperación permite exigir que cada tarjeta cite su documento y su licencia (CC BY-SA y ODbL piden atribución, que unos pesos diluirían). Tampoco hay datos etiquetados de itinerarios buenos y malos ni usuarios que los generen, y ajustar exigiría cómputo, un banco de evaluación que aún no existe (sección 11) y repetirlo con cada modelo base: desproporcionado para cuatro personas y 30 €/mes.

Lo que se ajusta en la práctica son los *prompts* (plantillas en `prompts.py`, con su *hash* en la traza), la escalera de filtros y los 8 candidatos por franja (`PLANNER_CANDIDATES`), la validación determinista y la temperatura: un único `CHAT_TEMPERATURE` de 0,7, injustificado para las salidas JSON (`brief`, `classify`, `ranking`), donde procede 0–0,3. El JSON se pide por *prompt* (esquema Pydantic en el `system`), sin *tool use*; si no valida hay una llamada de reparación y, si vuelve a fallar, el turno se degrada (mejores candidatos, títulos “Día N”) sin romperse. Un ajuste fino, o un modelo pequeño propio, solo se plantearía para tareas acotadas y verificables (extraer el brief, clasificar el mensaje) cuando las trazas (ADR 0024) reúnan miles de turnos reales etiquetables y el candidato supere a Haiku en el banco de evaluación con menos latencia y coste.

6.5 Orquestador PlanTrip

Orquestador. PlanTrip (ADR 0015, figura 4) es **sin estado**: cada `PlannerTurn` trae historial, brief e itinerario (solo ids), el `group_id` acuñado por el servidor codifica qué significa una selección y las tarjetas se rehidratan del almacén, nunca del cliente.

Grounding estricto. Cada candidato llega como una línea `id | nombre | categoría | distrito | tier | foto | resumen`; el modelo solo puede usar esos ids, con un *why* de menos de 140 caracteres sin precios. Los ids ajenos se **descartan** (contados en la traza) y los huecos se rellenan con los mejores candidatos. La tarjeta sale entera de metadatos, los precios son *tiers* (€ a €€€) y `strip_prices` borra todo importe con moneda del texto del modelo. Cada franja lanza cuatro búsquedas paralelas con una escalera de filtros (distritos del día y

Tabla 15: Llamadas LLM por turno (según el código, sin reparaciones); cada búsqueda añade una a Titan y una QueryVectors.

Turno	Llamadas
Primer mensaje incompleto	2
Brief completo → barrios	2
Barrio elegido → hoteles	1
Hotel elegido → borrador de N días	1 + N (≤ 8)
“Elige por mí” con brief completo	3 + N
Alternativas de una franja	1
Mensaje libre tras la estancia	2
Elegir o quitar una actividad	0

tier → ciudad y tier → ciudad), deduplica por título y antepone las que tienen foto.

Validación determinista. Una función pura avisa, por día, de actividades consecutivas a más de 3,5 km, exceso sobre el ritmo (3, 4 o 6 actividades), lugares cerrados según su horario y precios no verificados. El tiempo sale de Open-Meteo o de la normal del corpus; los vuelos, de un enlace sin precio.

Coste. Los días del borrador se generan en secuencia (tabla 15) para que el viaje tome forma en pantalla. A 1 y 5 USD por millón de tokens de entrada y salida (Haiku 4.5) se *estima* un borrador equilibrado de cuatro días en ~0,02 USD (~11 500 tokens de entrada, ~2 000 de salida) y un turno de una o dos llamadas en 0,002–0,006 USD. La traza (ADR 0024) registra tokens, coste y latencia por turno y la consola muestra p50/p95 (sección 11).

Observabilidad. La traza de cada turno (ADR 0024, sección 10) guarda modelo, *hash* del *prompt*, tokens, coste, latencia, *spans* y contadores propios de la IA (reparaciones, ids descartados, precios eliminados); de la identidad, solo el *subject*. Los tokens de embeddings aún no se contabilizan (TRA-232).

6.6 Valoración

Las métricas y pruebas se recogen en la sección 11. Cada tarjeta queda anclada a un documento recuperado; la prosa libre no. Debilidades. *Inyección de prompt*: el texto recuperado (Wikivoyage editable, OSM) entra en el *prompt* sin delimitarse como datos y las *og:image* son de terceros; la introducción española de Madrid llegó vandalizada al manifest. La única mitigación está en la red: *site_previews.py* solo pide URL *http(s)* de hosts con nombre público (sin IP literales, *localhost* ni *.local/.internal*) y revalida cada redirección contra SSRF. *Capacidad*: el límite real lo marcan las cuotas de Bedrock (no medidas), la concurrencia de Lambda y los 29 s de API Gateway en modo *buffered*; no hay pruebas de carga. Un solo proveedor y región: pasar a NVIDIA exige un *apply* manual y el ciclo de vida del modelo lo fija el proveedor. Además, un corpus 98,8 % en inglés ante usuarios hispanohablantes; la escalera reembebe la consulta en cada escalón, sin caché; y no hay reranker ni guardrails más allá de *strip_prices* (sección 12).

7 Arquitectura en la nube

El sistema vive en una cuenta de AWS (*eu-west-1*), sin VPC desde el 23 de septiembre de 2026. Todo está en Terraform (proveedor *hashicorp/aws* ~> 6.26, estado en S3 con *use_lockfile*) salvo el *bootstrap* (bucket de estado, proveedor OIDC y rol de CI, aplicado una vez con estado local) y el WAF, creado desde la consola. Figura 5 muestra la topología v3.

7.1 Topología v3

Borde. CloudFront sirve el export desde S3 privado (versionado, solo *Origin Access Control* con *SourceArn*) y reenvía */api/** sin caché: un único origen, sin CORS. Una CloudFront Function traduce */x/a /x/index.html*; */api/** va sin compresión para que el SSE llegue token a token.

Cómputo. API Gateway REST regional tiene dos *proxy*: */api/v1/{proxy+}* hacia *core-api* en modo *buffered* (límite de 29 s, de ahí el *timeout* de 30 s) y */api/v1/ai/{proxy+}* hacia *ai-api* en *STREAM* (hasta 15 min). Un *Dockerfile* construye ambas imágenes; Lambda Web Adapter [50] solo se activa bajo Lambda, y el contenedor corre igual en Compose (sección 8). *core-api* tiene 1769 MB (una vCPU); *ai-api*, que casi solo espera al modelo, 512 MB. Sin *provisioned concurrency*.

Datos e IA. DynamoDB *on-demand*: *core* (sección 5) e *interactions* (sección 10). Los vectores están en S3 Vectors [51] (índice *city-kb*, 1024 dimensiones, coseno) y los modelos en Bedrock con perfiles de inferencia EU: Claude Haiku 4.5 responde y Titan V2 embebe consultas; Nova Lite está declarado para títulos, pero el código no lo invoca (sección 6).

Identidad. Cognito, solo con Google y sin registro por contraseña (*allow_admin_create_user_only*), emite JWT RS256 (*ID* y *access* de 60 min, *refresh* de 30 días) con *code+PKCE* y cliente público. Terraform lee el JWKS en *plan* y lo inyecta como variable de entorno: validar un token no sale a la red. Sin VPC no hace falta NAT: las Lambdas salen a Internet por la red gestionada de AWS (Open-Meteo, Commons, *og:image*), y Bedrock, DynamoDB y S3 Vectors se autorizan por rol IAM.

7.2 Evolución v1 → v3

Tres iteraciones en ocho días (15–23 de septiembre; tabla 16) con un presupuesto real de 60 € para dos meses: la v2 costaba ~55 €/mes antes de resolver la salida a Internet (NAT ~35 €/mes o cinco *interface endpoints* ~40 €/mes). ECS no era requisito: Lambda sustituyó a Fargate a costa de no tener tareas en segundo plano ni estado en memoria entre peticiones.

7.3 Costes

Tabla 17 son estimaciones de los ADR (precios de lista, tráfico de demostración [52]): ~5 €/mes en recursos gestionados por Terraform (ADR 0023) más ~8 \$/mes del WAF creado a mano, unos 12–13 €/mes en total. No hay factura en esta memoria: la de septiembre debe sustituir la estimación.

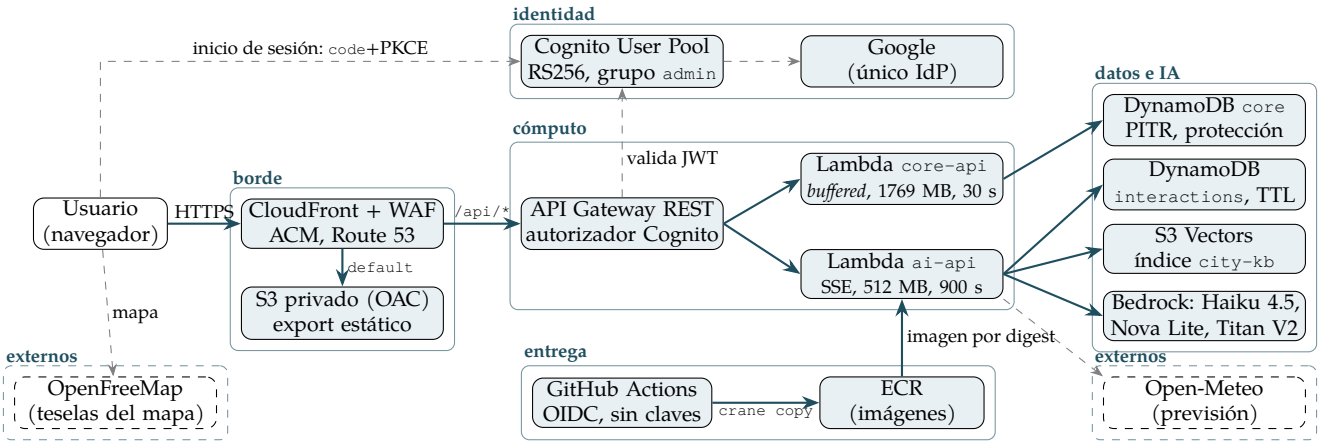


Figura 5: AWS v3 en eu-west-1, sin VPC. API Gateway exige un ID token de Cognito en todo método (incluidos los de salud). Ambas funciones reciben su imagen de ECR por digest (solo se dibuja la flecha a ai-api). Discontinuo: identidad y servicios externos. Fuente: `infra/aws/*.tf` y `docs/architecture/aws-architecture.drawio.svg`.

Tabla 16: Evolución de la arquitectura AWS.

Versión	Qué cambió	Por qué
v1 (ADR 0007)	Fargate + ALB + RDS; SSO local, OIDC en CI	Sin credenciales largas legibles por agentes
v2 (ADR 0008)	CloudFront origen único; API Gateway con <i>streaming</i> ; VPC Link + NLB; Bedrock	ALB HTTP bloqueaba Google Sign-In; CORS. NAT y NLB sin aplicar
v3 (ADR 0009)	Lambda + LWA; Cognito; sin NAT; VPC solo por RDS	30 €/mes; sin salida (tokeninfo, NVIDIA)
sin VPC (ADR 0023)	DynamoDB; RDS y VPC destruidos (TRA-219)	RDS: ~15 €/mes y la VPC

Tabla 17: Coste mensual estimado (€/mes) antes y después de retirar RDS.

Componente	ADR 0009	ADR 0023	Escala
Lambda ×2 (capa gratuita)	0	0	sí
API Gateway (3,50 \$/M pet.)	<1	<1	sí
CloudFront, S3, Route 53	~1	~1	>1 TB
Cognito (10 000 MAU gratis)	0	0	>10 000
RDS db.t4g.micro + 20 GB	~15 ^a	0	no
DynamoDB (2 tablas) + PITR	—	<1	sí
ECR, CloudWatch Logs, KMS	~1	~1	logs
Bedrock (tokens, demo)	1–3	1–3	sí
Total Terraform	~4 / ~19	~5	
WAF (consola, fuera de Terraform)		~8 \$	no
Total actual		~12–13	

^a0 € el primer año (capa gratuita).

7.4 Seguridad

Sin credenciales de larga duración (ADR 0007 prohíbe *access keys* en `.env`, `tfvars`, repositorio y contenedor): en local, IAM Identity Center (`just aws-login`); en CI, OIDC con un rol de 1 h que solo confía en el entorno `aws`. Roles mínimos: `core-api` sin `Scan`; `ai-api` con sus trazas, un índice de vectores de solo lectura y Bedrock limitado a perfiles EU. CI parte de `PowerUserAccess` con IAM acotado a `travel-ai-*`. PITR y *deletion protection* cubren `core` y el `user pool`; el WAF, tres grupos de reglas gestionadas. Contra SSRF, `site_previews.py` solo pide `og:image` a `hosts` con nombre público y revalida cada redirección (sección 6). Carencias: sin *usage plans* ni *throttling*, cualquier usuario autenticado puede generar gasto ilimitado en Bedrock;

el estado de Terraform contiene dos secretos (clave NVIDIA y secreto OAuth de Google); en local se usa `AdministratorAccess` (la organización no permitió un *permission set* acotado); no hay GuardDuty, Config ni Security Hub.

7.5 Raíces del dominio y la alternativa GCP

ADR 0010 importó al estado, con `prevent_destroy`, la zona de Route 53 y el certificado ACM (us-east-1, como exige CloudFront), antes *data sources*: un `apply` reconstruye el borde sin exponer recursos irreversibles a un `destroy`. `infra/gcp/` solo pasa `terraform validate` en CI y, según su README, no está portado a DynamoDB, así que no despliega `core-api`.

7.6 Valoración

Cada ADR corrigió al anterior con cifras de la cuenta real y el coste fijo estimado pasó de ~55 € a ~12–13 €/mes (~5 € en Terraform más el WAF). Deuda: WAF fuera de IaC y del presupuesto; una sola región, sin *staging*; los despliegues fijan el digest sin alias ni tráfico ponderado; arranques en frío de 1–3 s sin mitigar; y *interactions* no tiene PITR ni protección contra borrado: coherente para un log con TTL, pero deja sin copia el único registro de lo que hizo la IA. `infra/gcp/` debe retirarse o portarse (sección 12).

8 Entorno de desarrollo

Como desarrollan a la vez una persona y varios agentes con *shell*, el *devcontainer* [53] (base Ubuntu de Microsoft) instala las versiones de CI: Python 3.12 con `uv`, Node.js 24, `just`, Terraform 1.16 (*build arg*), AWS CLI v2, `crane` 0.22.1, `gh`, `ripgrep/jq`, Claude Code y, opcionales, Codex, Gemini y Copilot CLI. `post-create.sh` copia un perfil SSO de ejemplo y ejecuta `just setup` (cuatro `.env`, `uv sync`, `npm install`) y Chromium de Playwright. Compose añade DynamoDB Local y volúmenes que conservan `.venv`, `node_modules`, la configuración de los agentes y `~/aws`. AWS se usa por IAM Identity Center (`just aws-login`), sin claves de larga duración en disco que un agente pudiera leer (ADR 0007). Se descartaron Docker-in-Docker, SAM y CDK; el *stack* de Compose se lanza desde el *host* porque monta rutas relativas a su sistema de ficheros.

Tabla 18: Recetas de `just` agrupadas por uso (selección de las 41).

Grupo	Recetas
Desarrollo	<code>setup</code> , <code>dev-{core, ai, frontend}</code> , <code>dynamodb-local</code>
Calidad	<code>lint</code> , <code>test{, -core, -ai, -common, -corpus, -frontend}</code> , <code>test-e2e{, -static, -stack}</code> , <code>docs-check</code> , <code>contracts{, -check}</code>
Corpus	<code>corpus-discover</code> , <code>corpus</code> , <code>corpus-report</code> , <code>index</code>
Stack y nube	<code>docker-up</code> , <code>build-stack</code> , <code>stack-up</code> , <code>aws-login</code> , <code>infra-{validate, fmt-check}</code> , <code>release</code>

Tabla 19: Paridad entre el entorno local y AWS.

Aspecto	Local frente a AWS	Paridad
Cómputo	misma imagen; <code>uvicorn</code> / <code>Lambda Web Adapter</code>	alta
Origen	<code>nginx :8080</code> / <code>CloudFront</code>	alta
Datos	<code>DynamoDB Local</code> o <code>moto</code> / <code>DynamoDB</code>	alta
LLM	<code>NVIDIA NIM</code> con clave / <code>Bedrock</code> por rol IAM	media
Auth	<code>AUTH_MODE=local</code> (JWT HS256 por CLI) / <code>Cognito RS256</code>	media
Vectores	sin emulador: índice real de S3 Vectors, requiere SSO	baja
WAF	sin equivalente local	nula
WebGL	<code>SwiftShader</code> sin interfaz; quitar <code>WAYLAND_DISPLAY</code> (TRA-180)	aviso

Las 41 recetas del `justfile` (tabla 18) son la única interfaz para personas, agentes y los `jobs` de `pr.yml`. Con `dynamodb-local` (`moto`, `:8002`), `tests` y `desarrollo` no necesitan Docker. `stack-up` sirve `export`, APIs y `DynamoDB Local` en un origen (`:8080`) tras un `nginx` que reproduce `CloudFront` (SSE sin `buffering`, `/api/` a `core_api`, índices y `404.html`); lo ejecuta el `job e2e-stack` (tabla 19).

Valoración. Un agente recién arrancado ejecuta las comprobaciones de CI sin más instrucciones, y cada PR prueba el `export` como lo sirve `CloudFront`. La paridad cae donde hay un servicio gestionado sin emulador: el RAG local depende de AWS real y no funciona sin conexión, y el WAF solo se comprueba en producción. El `stack` completo no corre dentro del `devcontainer`, que arrastra herramientas sin uso (el `plugin` de `Sesion Manager` de la etapa `Fargate`).

9 CI/CD y proceso de entrega

Cada cambio empieza en un `issue` de `Linear` y llega a producción por dos caminos (figura 6): el `frontend` se despliega al fusionar; el `backend`, a mano.

9.1 Integración continua

`pr.yml` corre en cada PR contra `main`. Un `job` inicial evalúa con `dorny/paths-filter` diez filtros de ruta (uno por paquete, más `stack`, `infra` y `docs`) y cada `job` depende del suyo: un cambio de documentación no ejecuta `pytest` ni `Playwright`. La tabla 20 resume los niveles de prueba: `contracts-check` regenera el OpenAPI y los tipos `TypeScript` y falla si difieren de lo comprometido, y `e2e-stack` ejecuta `Playwright` sobre el `stack` de `Compose` (sección 8) con una sesión generada en `core_api`. CI construye además cada imagen, sin `provenance` ni `sbom`, para comprobar el `Lambda Web Adapter` sin publicarla. `Terraform` solo pasa `fmt`

Tabla 20: Plan de pruebas. Casos ejecutados con `just test` el 23-09-2026, todos en verde (`travel_common` 37, `core_api` 199, `ai_api` 597, `city_corpus` 254; muchos tests están parametrizados). L: local.

Nivel	Herramienta	Dónde	Casos / cobertura
Unitario	<code>pytest</code> ; <code>Vitest</code>	L, CI	1 087 backend; 747 frontend
Integración	<code>pytest</code> , <code>moto</code> , LLM guionizado	L, CI	incluidos en los 1 087
Contrato	<code>contracts-check</code>	CI	<i>drift</i> OpenAPI y tipos
E2E	<code>Playwright</code> (<code>export</code> , <code>Compose</code>)	CI	9 specs, no ejecutados aquí
Modelo real	<code>planner-smoke</code> (NVIDIA)	L	manual; no contra <code>Bedrock</code>
Seguridad	reglas de <code>ruff</code>	L, CI	estático; sin DAST ni <i>pentest</i>
Carga, a11y	—	—	no existen (a11y: solo revisor UX)

y `validate`: ninguna PR ejecuta un plan.

9.2 Despliegue

Al fusionar, `backend-images.yml` publica `core-api` y `ai-api` en GHCR, etiquetadas con el SHA completo. `deploy-backend.yml` solo se lanza a mano, en el entorno `aws` de GitHub, que guarda los dos únicos secretos reales (clave de NVIDIA y secreto del cliente de Google); entra en AWS por OIDC [54], sin claves, copia las imágenes a ECR con `crane copy` [55] y pasa el `digest` a `Terraform`. Se descartó `docker buildx imagetools create` porque envuelve la imagen en un índice OCI que `Lambda` rechaza, y `crane` no necesita `daemon` (TRA-133); el `digest`, a diferencia de una etiqueta mutable, obliga a `Lambda` a redespargar. El `apply` es manual (`apply=false` por defecto): solo hay un entorno, sin `staging`, y un secreto ausente llega a `Terraform` como cadena vacía y borra la configuración viva (lo advierte el propio `workflow`), así que el `runbook` exige revisar antes un `plan` que solo cambie los dos `image_uri`. El `frontend` (`deploy.yml`) no tiene esa puerta: cada `push` a `main` que toca `src/frontend/**` se sincroniza con S3 e invalida `CloudFront`.

`Dependabot` abre cada semana PRs agrupadas y retiene, con el motivo escrito, tres versiones mayores (TRA-195).

Entregables: imágenes Docker y scripts de despliegue.

Un único `src/backend/Dockerfile` multi-etapa produce las dos imágenes con el `build arg SERVICE`: `uv sync --frozen` instala solo ese servicio y `travel_common`, y la etapa final (`python:3.12-slim`, usuario sin privilegios, `HEALTHCHECK`) lleva el `Lambda Web Adapter` como extensión, activa solo bajo `Lambda`; fuera, `docker/entrypoint.sh` arranca `uvicorn`. La misma imagen corre así en `Compose` y en `Lambda`. `docker-compose.yml` define `proxy` (`nginx`, `docker/nginx.conf`), `core_api`, `ai_api` y `DynamoDB Local`. El `frontend` no tiene imagen: es un `export` estático, sin proceso servidor, que sirven `nginx` o `S3+CloudFront`. Se arranca con `just setup`, `docker-up` y `stack-up`, y se despliega con los `workflows` y `terraform apply` (`runbooks` `deploy.md` y

Tabla 22: Dependencias externas y su efecto en la disponibilidad.

Dependencia	Modo de fallo	Mitigación actual
Bedrock (Haiku, Titan)	Caen chat, planificador y recuperación	3 intentos (1 y 2 s) antes del primer <i>token</i> ; NVIDIA manual (<i>apply</i>)
DynamoDB <i>core</i>	Cuentas, viajes y chats inaccesibles	Multi-AZ gestionado; PITR
DynamoDB <i>interactions</i>	Se pierden trazas	<i>Best effort</i> ; nunca rompe la respuesta
S3 Vectors	Sin candidatos para el planificador	Ninguna; <i>just index</i> lo reconstruye
Cognito	Sin sesión; la API lo rechaza todo	Ninguna
Google	Sin nuevos inicios de sesión	Ninguna (único IdP)
Open-Meteo	Sin previsión	<i>Timeout</i> de 5 s; normales climáticas del corpus
OpenFreeMap	Mapa sin teselas	Ninguna; el planificador sigue
GitHub / GHCR	No se construye ni despliega	Producción intacta: Lambda lee de ECR

SSE) con tokens, coste según una tabla de precios versionada y latencia por paso, en vocabulario de Open-Telemetry GenAI, enlazada a usuario, sesión y viaje: se recorre petición → recuperación → modelo → eventos → respuesta. Es *best effort* y se escribe antes del [DONE], porque Lambda puede congelar el proceso al cerrarse el *stream*. (3) *Rendimiento*: la consola /admin/ agrega por día turnos, errores, tokens, coste, latencia p50/p95, primer evento, usuarios y sesiones, e inspecciona cada turno. La traza guarda texto libre (mensaje, historial y respuesta) y no se borra al suprimir la cuenta: un problema de privacidad (RGPD) pendiente (sección 12). Los *endpoints* de salud sirven de *readiness* a Lambda Web Adapter, pero quedan tras el autorizador de Cognito.

No existe. Ni alarmas de CloudWatch, SNS, AWS Budgets, dashboards, X-Ray ni colector OTel, ni comprobación externa de disponibilidad (imposible sin credenciales), ni SLO ni presupuesto de errores, ni *smoke test* contra producción tras desplegar: *e2e-stack* prueba Compose en CI y *deploy.md* solo pide comparar el *digest*. Una caída o un pico de gasto pasan inadvertidos hasta que alguien los ve.

10.2 Dependencias y alta disponibilidad

La disponibilidad (tabla 22) descansa en servicios gestionados multizona, no en configuración propia. Todo está en *eu-west-1*; *us-east-1* solo aloja el certificado de CloudFront. Sin RPO/RTO escritos; deducidos del diseño, en *core* el RPO es casi continuo por PITR (35 días), pero el RTO no se ha medido ni ensayado; *interactions* no tiene copia; los vectores tienen RPO igual al último *just index* y RTO de minutos. El resto sale de Git y Terraform, salvo el WAF, que se recrearía a mano. No hay *runbook* de DR.

10.3 Costes

El coste fijo estimado es de unos 5 €/mes en recursos gestionados por Terraform (ADR 0023; tabla 17) más ~8 €/mes del WAF creado a mano: unos 12–13 €/mes en total. Es una estimación; la factura de septiembre de-

Tabla 23: Métricas del proyecto (líneas y recuentos, 23-09-2026).

Métrica	Valor
Commits desde 10-03-2026 / sept.	324 / 187
PR fusionadas / sept.	181 / 145
Issues TRA citadas en commits	107
Python: backend y tools (274 arch.)	43774
T5/TSX: frontend (259 arch.)	34623
Terraform / YAML / Markdown	2560 / 1019 / 8469
Tests backend ejecutados	1087
Tests frontend ejecutados / e2e	747 / 9
ADR / <i>runbooks</i> / recetas <i>just</i>	23 / 7 / 41

Recuentos con *git*, *gh* y *wc*, sin código generado ni vendorizado; tests ejecutados con *just test* (todos pasan; desglose en la tabla 20); los 9 *specs* e2e no se ejecutaron.

be sustituirla. El variable lo domina Bedrock. Con Haiku 4.5 (1/5 \$ por millón de tokens de entrada/salida), un borrador de cuatro días se estima en ~0,02 \$ y un turno de una o dos llamadas en 0,002–0,006 \$ (sección 6). El coste real queda en cada traza, pero no se ha agregado, y los tokens de embeddings no se contabilizan (TRA-232). Mil borradores al mes costarían unos 20 \$, más que la infraestructura, y API Gateway no limita el uso por cliente: es el principal riesgo de gasto.

10.4 Propuesta mínima

Tres medidas casi gratuitas: AWS Budgets con aviso al 80 % de 30 €/mes; alarmas SNS sobre errores y *throttles* de Lambda y 5xx de API Gateway; y una ruta de salud pública sin datos para un comprobador externo. Y dos SLO iniciales: disponibilidad mensual del API del 99,5 % (unas 3,6 h de presupuesto de errores al mes) y p95 del primer evento SSE del planificador por debajo de 3 s, que ya mide la traza (sección 12).

11 Evaluación de la solución

La tabla 24 reúne lo que hoy se mide del sistema, con su origen y su grado de evidencia: *medido* (instrumentado y reproducible), *ejecutado* (pruebas que pasan o fallan), *estimado* (cálculo sobre precios de lista) o *en CI* (se ejecuta en cada PR, no para esta memoria). Las cifras de recuperación, latencia y coste proceden del spike TRA-151 y se detallan en las tablas 12 a 14; aquí no se repiten sus alternativas.

Lectura. La evidencia es sólida en lo estructural (1087 tests backend y 747 frontend en verde, contratos y lint que bloquean el *merge*, un gate que impide indexar una ciudad incompleta) y débil en lo que el usuario percibe. La recuperación se midió una vez, en una ciudad y con 32 consultas: sirvió para decidir el almacén, no para certificar la calidad. Los costes por turno son estimaciones sobre el código; la consola ya registra el coste real, pero sin agregar por viaje ni publicar.

Lo que no está evaluado. La calidad del itinerario extremo a extremo (coherencia, distancias, interés), sin juez LLM ni humano; el comportamiento con usuarios reales, que no los hay; la carga y las cuotas de Bedrock; la seguridad más allá del análisis estático (inyección de *prompt*, sin DAST ni *pentest*); la accesibilidad, sin au-

Tabla 24: Métricas técnicas y funcionales. Fechas de 2026. Spike: docs/architecture/vector-store-spike.md; ai: src/backend/services/ai_api; cc: src/backend/tools/city_corpus.

Métrica	Valor	Obtención	Origen y límites
<i>Técnicas</i>			
R@5 / R@10 / MRR	0,688 / 0,812 / 0,591 (S3 Vectors)	medido, 17-09	Spike: 32 consultas, 52 ids esperados, solo Budapest, 4 en español, sin intervalos de confianza
Acuerdo con coseno exacto	0,969 del top-10	medido, 17-09	Spike; mide el almacén, no la relevancia
Latencia búsqueda (caliente)	p50 64 ms, p95 69 ms (2 048 MB); frío 91 ms	medido, 17-09	Spike, 128 consultas desde Lambda en eu-west-1
Latencia embedding Titan	~75 ms; búsqueda total p50 137 ms	medido, 17-09	Spike; despreciable frente al LLM
Coste de recuperación	~2,5 USD por millón de consultas; indexar Budapest, 0,012 USD	estimado / medido	Spike; precios de us-east-1
Coste LLM turno / borrador	0,002–0,006 USD / ~0,02 USD (4 días)	estimado	Tokens según el código y precios de Haiku 4.5 (tabla 15)
Tests backend	37 / 199 / 597 / 254 (common, core, ai, cc), en verde	ejecutado, 23-09	just test; LLM guionizado, DynamoDB con moto (tabla 20)
Tests frontend	747 (Vitest), en verde	ejecutado, 23-09	just test-frontend
E2E	9 specs de Playwright	en CI	src/frontend/e2e; no ejecutados para esta memoria
Lint y contratos	ruff, pyright, eslint, drift de OpenAPI y tipos	en CI	.github/workflows/pr.yml; bloquean el merge
<i>Funcionales</i>			
Gate de readiness	11 umbrales; pasan Berlín, Budapest y Madrid; Bolonia no (3 hoteles con foto de 10)	ejecutado, 22-09	cc/config/readiness.py; data/<ciudad>/report.json
Sesión con modelo real	7 turnos; falla si una actividad no tiene foto o una tarjeta lleva precio	manual	just planner-smoke; Nemotron y recuperador por palabras, no Bedrock; sin registro de resultados
Flujos de usuario (e2e)	brief → 3 días con hotel → cambio de franja; reabrir por URL; renombrar y borrar viajes; consola admin; móvil; HTML prerenderizado	en CI	planner, trips, admin*, mobile, prerender, smoke; LLM sustituido por una sesión grabada
Modo demo	sesión grabada de Budapest, con aviso, si falta la ruta del planner	en CI	planner.spec.ts
Validación de itinerarios	avisos por día: saltos >3,5 km, ritmo, cerrados, precios	en cada turno	Función pura en ai; tests unitarios
Consola /admin	latencia p50/p95, p50 del primer evento, tokens, coste, usados/recuperados	medido (prod.)	Trazas de ADR 0024; sin análisis publicado ni tokens de embeddings (TRA-232)

ditoría; y el español, con un corpus 98,8 % en inglés y solo 4 consultas de cruce. El análisis y las propuestas están en la sección 12.

Criterios de aceptación propuestos. Coherentes con la tabla 25, se propone que bloqueen el *merge* o el despliegue: $R@10 \geq 0,75$ en un banco de 100 consultas por ciudad con un 30 % en español; una nota media del juez LLM $\geq 4/5$ sobre itinerarios, validada con $\kappa \geq 0,6$ frente a 50 itinerarios anotados; p95 del primer evento < 3 s, lo que exige añadir ese percentil a la consola (hoy solo da el p50); y coste p95 por viaje $\leq 0,05$ USD, una vez contabilizados los embeddings.

12 Análisis crítico y propuestas de evolución

Esta sección ordena por ámbito las valoraciones repartidas por la memoria y las convierte en propuestas priorizadas con una métrica de éxito.

12.1 Lo que sostiene el proyecto

Las decisiones de infraestructura se apoyaron en cifras. ADR 0014 eligió S3 Vectors por simplicidad operativa y el *spike* TRA-151, confirmatorio, construyó los dos almacenes: con 32 consultas de una ciudad (4 en español) no hubo diferencia de calidad apreciable, sin intervalos de confianza, y el arranque en frío favoreció a S3 Vectors (sección 6). Las tres versiones de la nu-

be y la retirada de RDS se justificaron con cifras de la cuenta real (ADR 0009, ADR 0023; sección 7), y la política de fotos prefiere no mostrar ninguna antes que una equivocada (ADR 0021, ADR 0022). El anclaje es estructural: toda tarjeta procede de un documento recuperado del corpus, los ids no recuperados se descartan y los precios son niveles, nunca importes; la prosa libre del modelo (respuestas de chat, el *why* de 140 caracteres) sí puede mencionar lugares y no está verificada. El asistente que Expedia retiró respondía sin inventario ni precios reales [13]; Kyrian aporta anclaje documental (existencia, descripción, fuente), pero no disponibilidad ni precio, que exigen integrar inventario (horizonte de 12 meses; sección 3). Completan la base un coste fijo estimado de unos 12–13 €/mes (5 € en Terraform más ~8 \$ del WAF manual; sección 10), contratos verificados en CI, trazas de turno (ADR 0024), un corpus con atribución y un proceso trazable de issue a PR y a ADR (sección 9).

12.2 Lo que se haría distinto

Producto y experiencia. El producto se construyó sin validación con usuarios: no hay entrevistas, analítica de uso ni valoración por tarjeta. Lo delatan el recorte a una ciudad por viaje (ADR 0019) y “Mis viajes”, añadido porque el login dejaba al usuario ante un planner vacío (ADR 0020). El viajero no puede corregir a la IA (TRA-146); la tarjeta no enseña la fuente ni el fragmen-

to recuperado, que son el diferenciador, y su contenido sale en inglés; el mapa une paradas con líneas rectas, sin tiempos (sección 2). La accesibilidad está declarada, no medida.

Inteligencia artificial. Tres huecos pesan más que el resto:

- *Sin evaluación extremo a extremo.* TRA-148 no tiene código: los 597 tests de `ai_api` comprueban el orquestador con un LLM guionizado, no si un itinerario es bueno. No hay juez LLM [61], ni regresión de *prompts* en CI, ni guardrails: el texto de Wikivoyage, editable por cualquiera, entra en el *prompt* y abre un vector de inyección indirecta [62].
- *Lo medido no está en producción.* La fusión BM25+RRF subió el MRR de 0,591 a 0,651 en el *spike*, pero las tarjetas se sirven con búsqueda densa sola (tabla 12).
- *Lo decidido no está implementado.* La política de corpus (TRA-210: ≤ 2000 documentos por ciudad, cuotas y puntuación de calidad) sigue en papel: Berlín tiene 12 240 documentos, un 53 % de restaurantes sin descripción, y Bolonia 3 hoteles con foto de 118 (tabla 11). Los *gates* cuentan cantidades, no calidad: la introducción vandalizada de Madrid y sus guías de distrito vacías los superaron.

El corpus está en un 98,8 % en inglés para un público hispanohablante, y el cruce de idiomas se midió con 4 de las 32 consultas. Hay además deuda de ingeniería: JSON pedido por *prompt* con temperatura 0,7 y una llamada de reparación en lugar de *tool use* [63]; un único modelo, sin *failover* automático; una escalera de filtros que reembebe la misma consulta, sin caché; y tokens de embeddings sin contabilizar (TRA-232).

Arquitectura de software. El doble modo de autenticación (HS256 local, RS256 con Cognito) se ha vuelto permanente; `ai_api` deja 60 min sin revocación (sección 5). Sin límites de uso por usuario, una cuenta comprometida o un bucle del cliente es el mayor riesgo económico real de un sistema de ~ 12 €/mes. La traza se escribe en el camino crítico (~ 100 ms) en una tabla sin PITR, guarda texto libre 90 días y no se borra con la cuenta. La documentación deriva: `code-quality-review.md` y ADR 0013 son anteriores a DynamoDB y `CLAUDE.md` exige un MCP de Pencil no configurado.

Nube, operación y proceso. No hay alarmas, presupuesto de AWS, comprobación externa, *staging* ni despliegue gradual; el WAF está fuera de Terraform y el estado guarda dos secretos (secciones 7 y 10): la observabilidad de negocio va por delante de la operativa. El *bus factor* es 1, los revisores son modelos con sesgos compartidos y el coste en *tokens* no se mide; las 145 PR de septiembre (tabla 23) superan la capacidad de validar lo entregado.

12.3 Propuestas priorizadas

La tabla 25 agrupa las propuestas en tres horizontes. Esfuerzo e impacto son estimaciones cualitativas; cada

fila une la carencia que la justifica con su criterio de éxito.

12.4 Hoja de ruta

El orden sigue un criterio explícito. El primer mes acota el riesgo económico, operativo y legal: presupuesto, alarmas, límites de uso y privacidad cuestan horas y cierran las vías por las que el proyecto puede hacerse caro o ilegal de golpe. Los tres meses siguientes miden la calidad de la IA antes de mejorarla: sin banco de evaluación no se puede saber si la búsqueda híbrida, un *reranker* o un modelo mayor mejoran algo. Solo entonces tiene sentido ampliar el producto y probar el negocio con un piloto B2B, que además daría los primeros usuarios reales.

13 Conclusiones y trabajo futuro

Kyrian World funciona en producción con un coste fijo estimado de ≈ 12 €/mes (≈ 5 €/mes en Terraform más el WAF creado a mano; sin factura real todavía) y anclaje estructural de sus tarjetas; su calidad de itinerario, su aceptación por usuarios y su viabilidad económica siguen sin medir. Un equipo de cuatro personas, con agentes de IA y un proceso escrito, lo llevó a producción en seis meses para cuatro ciudades. El anclaje tiene un alcance preciso: toda tarjeta procede de un documento recuperado y los ids no recuperados se descartan, pero la prosa libre del modelo (respuestas de chat, el *why* de 140 caracteres) puede mencionar lugares no verificados. El *spike* de vectores confirmó S3 Vectors, elegido antes por simplicidad operativa (ADR 0014): sin diferencia de calidad apreciable en 32 consultas de una ciudad.

No hay usuarios reales, así que no se sabe si el viajero valora la verificabilidad lo bastante como para preferirla a un chat generalista gratuito, y la viabilidad económica descansa en hipótesis (sección 3). El trabajo futuro, priorizado, está en la sección 12; antes de ponerlo delante de usuarios de pago habría que hacer cuatro cosas:

1. Acotar el riesgo: límites de uso por usuario, presupuesto y alarmas.
2. Medir la calidad: un banco de evaluación extremo a extremo en CI que bloquee regresiones.
3. Cumplir en privacidad: publicar la política y los términos y borrar las trazas al suprimir la cuenta.
4. Validar con usuarios: valoración por tarjeta y un piloto con una DMO que aporte uso real y un primer ingreso.

El vídeo demo opcional que contempla el guion no se incluye en esta entrega.

Referencias

- [1] Open-Meteo. Open-Meteo weather forecast API. <https://open-meteo.com/en/docs>, 2026. Consultado el 23-09-2026.
- [2] MapLibre contributors. MapLibre GL JS. <https://maplibre.org/maplibre-gl-js/docs/>, 2026. Consultado el 23-09-2026.
- [3] OpenFreeMap. OpenFreeMap: free and open-source map hosting. <https://openfreemap.org/>, 2026. Consultado el 23-09-2026.
- [4] W3C. Web content accessibility guidelines (WCAG) 2.2. <https://www.w3.org/TR/WCAG22/>, 2023. W3C Recommendation. Consultado el 23-09-2026.
- [5] Vercel. Next.js documentation: Static exports. <https://nextjs.org/docs/app/guides/static-exports>, 2026. Consultado el 23-09-2026.

Tabla 25: Propuestas priorizadas. Esf.: esfuerzo; Imp.: impacto (B bajo, M medio, A alto); Hor.: horizonte en meses. Tras “▷”, la métrica de éxito.

Propuesta	Ámbito	Evidencia / métrica de éxito	Esf.	Imp.	Hor.
1 mes: proteger lo que hay					
AWS Budgets al 80 % y alarmas CloudWatch→SNS (5xx, duración, <i>throttles</i> , coste de Bedrock) [64]	Plataforma	Hoy un pico de gasto o una caída solo se ven si alguien mira (sección 10) ▷ 0 alertas sin atender en 24 h	B	A	1
<i>Throttling</i> y <i>usage plan</i> en API Gateway [65]; cuota diaria por usuario; contabilizar embeddings (TRA-232) y agregar el coste por turno	Plataforma	Bedrock sin límite por usuario; coste en trazas nunca agregado ▷ coste p95 por viaje ≤0,05 \$	B	A	1
Ruta de salud pública, comprobador externo y <i>smoke test</i> al final de <i>deploy-backend.yml</i>	Plataforma	Salud tras Cognito; el primer contacto con producción es el despliegue ▷ disponibilidad ≥99,5 %; <i>smoke</i> verde en cada despliegue	B	M	1
Privacidad y términos publicados, borrado de trazas al suprimir la cuenta (RGPD), aviso de IA (art. 50) y atribución CC BY-SA visible en cada tarjeta	Legal	El login enlaza políticas que no existen; trazas 90 días ▷ test de borrado en CI; 100 % de tarjetas con fuente	B	A	1
WAF a Terraform; secretos fuera del estado; actualizar <i>code-quality-review.md</i> , ADR 0013 y la regla de Pencil	Plataforma	WAF manual (~8 \$/mes); dos secretos en el estado ▷ <i>plan</i> sin recursos fuera de Terraform	B	M	1
3 meses: medir y mejorar la calidad					
Banco de evaluación en CI (TRA-148): 100 consultas por ciudad, recall/MRR y juez LLM sobre itinerarios, con umbrales que bloquean el <i>merge</i>	IA	597 tests no miden calidad; eval solo de Budapest ▷ $R@10 \geq 0,75$ (30 % en español); juez $\geq 4/5$ y $\kappa \geq 0,6$ frente a 50 itinerarios anotados	M	A	3
Mitigar la inyección: contenido recuperado delimitado como datos, filtro de instrucciones al construir el corpus, salida libre sin URL	IA	Wikivoyage editable; intro de Madrid vandalizada ▷ 0 éxitos en el test adversarial del banco	M	A	3
Política de corpus TRA-210 con puntuación de calidad y detector de vandalismo (edad y diff de la revisión)	IA	Berlín 6,1 veces el tope ▷ ≤2000 documentos por ciudad	M	A	3
Corpus bilingüe (traducción o embeddings evaluados en español): el segmento es hispanohablante	IA	98,8 % en inglés; 4 consultas en español ▷ $R@10$ en español a 0,05 del inglés	A	A	3
Híbrido BM25+RRF para tarjetas; embeber la consulta una vez y cachear; <i>reranker</i> [66] solo si el banco lo justifica	IA	MRR 0,591→0,651 (tabla 12) ▷ MRR ≥0,65 sin perder $R@10$	M	M	3
<i>Tool use</i> [63]; temperatura 0-0,3 para JSON; Sonnet para el ranking solo si el banco compensa su coste (2-3×)	IA	JSON por <i>prompt</i> y reparación ▷ 0 llamadas de reparación	M	M	3
Pruebas de carga con k6 [67] o Locust [68] contra Compose con LLM simulado y prueba corta real contra Bedrock	Plataforma	Sin pruebas de carga; cuotas de Bedrock sin medir ▷ usuarios concurrentes antes de <i>throttling</i> ; p95 primer evento <3 s	M	A	3
Edición manual y guardado (TRA-146); fuente y fragmento en la tarjeta, en el idioma del usuario; valoración por tarjeta ligada a la traza	UX	El viajero no puede corregir a la IA ni ver la fuente ▷ $\geq 4/5$ de valoración media	A	A	3
Auditoría axe [69] y Lighthouse en CI	UX	WCAG 2.2 declarado, no medido ▷ Lighthouse a11y ≥95	B	M	3
<i>Staging</i> por <i>workspace</i> de Terraform y alias de Lambda con <i>canary</i> [70]	Plataforma	Un solo entorno ▷ 0 <i>apply</i> sin <i>plan</i> probado en <i>staging</i>	M	M	3
12 meses: ampliar producto y negocio					
Piloto B2B con una DMO en marca blanca, sobre <i>/add-city</i>	Negocio	LTV/CAC B2B de 6-12 frente a ≈0,5 en B2C (sección 3) ▷ un contrato firmado	A	A	12
Viajes multiciudad; rutas y tiempos entre paradas con OSRM [71] o Valhalla [72]; exportar a PDF/ICS y compartir	UX	ADR 0019; líneas rectas; sin salida del producto ▷ tiempo entre paradas en el 100 % de los días	A	A	12
Agente con herramientas para datos vivos (horarios, precios, disponibilidad): conserva la procedencia y solo muestra un importe con fuente en vivo citada, la excepción a “nunca un número” de ADR 0018	IA	Sin inventario ni precios (el fallo de Romie) ▷ 100 % de importes con fuente	A	A	12
Afiliación medida (Booking, GetYourGuide) y plan comercial de Open-Meteo	Negocio	0,15-0,76 € por viaje, hipotético; Open-Meteo no comercial [31] ▷ ingreso por viaje medido	M	M	12
Revisión humana muestral de PR y coste por ola; retirar o portar <i>infra/gcp</i> ; segunda región solo si un SLO lo exige	Proceso	<i>Bus factor</i> 1; <i>tokens</i> sin medir ▷ 10 % de PR revisadas por una persona	B	M	12

- [6] UN Tourism. International tourist arrivals up 4 % in 2025 reflecting strong travel demand around the world. <https://www.untourism.in/t/news/international-tourist-arrivals-up-4-in-2025-reflecting-strong-travel-demand-around-the-world>, 2026. Consultado el 23-09-2026.
- [7] Instituto Nacional de Estadística. Encuesta de gasto turístico (EGATUR). diciembre 2025. datos provisionales. <https://www.ine.es/dyngs/PreNSA/EGATUR1225.htm>, 2026. Consultado el 23-09-2026.
- [8] Skift. Online travel is set for growth, but AI threatens the status quo. <https://skift.com/2025/12/23/online-travel-is-set-for-growth-but-ai-threatens-the-status-quo/>, 2025. Consultado el 23-09-2026.
- [9] YouGov España. La IA entra en la maleta: así se transforma la planificación de viajes en España con las nuevas tecnologías. <https://yougov.com/es-es/articulos/54365-la-ia-entra-en-la-maleta-asi-se-transforma-la-planificacion-de-viajes-en-espana-con-las-nuevas-tecnologias>, 2026. Consultado el 23-09-2026.
- [10] Adido Digital. A round up of Phocuswright Europe 2025. <https://www.adido-digital.co.uk/travel-trends/phocuswright-europe-2025/>, 2025. Consultado el 23-09-2026.
- [11] Travel Professional News. New Phocuswright research: European travelers are using AI more, but trusting it less than friends, family and human advisors. <https://travelprofessionalnews.com/new-phocuswright-research-european-travelers-are-using-ai-more-but-trusting-it-less-than-friends-family-and-human-advisors>, 2026. Consultado el 23-09-2026.
- [12] TechEdgeAI. AI travel planning is outpacing DMO readiness (estudio de Mindtrip, Sabre, Sojern y MMGY). <https://techedgeai.com/travelers-are-outpacing-destination-marketers-on-ai-adoption/>, 2026. Consultado el 23-09-2026.
- [13] Hotel News Resource. Expedia group details AI lessons learned from early chatbot efforts. <https://www.hotelnewsresource.com/article141624.html>, 2026. Consultado el 23-09-2026.
- [14] MonkeyTravel. Layla AI review 2026: Pricing, features & the best free alternative. <https://monkeytravel.app/blog/layla-ai-review-2026>, 2026. Consultado el 23-09-2026.
- [15] Skift. Expedia acquired AI trip-planner Layla: Exclusive. <https://skift.com/2026/07/31/expedia-acquired-ai-trip-planner-layla-exclusive/>, 2026. Consultado el 23-09-2026.
- [16] Pulse 2.0. Mindtrip: New strategic investments raised from Capital One Ventures and United Airlines Ventures. <https://pulse2.com/mindtrip-strategic-investments/>, 2025. Consultado el 23-09-2026.
- [17] Wikipedia. Guidegeek. <https://en.wikipedia.org/wiki/GuideGeek>, 2026. Consultado el 23-09-2026.
- [18] Skift. ChatGPT travel apps are now a thing – starting with Booking.com and Expedia. <https://skift.com/2025/10/06/expedia-booking-chatgpt-apps-openai/>, 2025. Consultado el 23-09-2026.
- [19] 9to5Google. Google AI mode will help you create travel plans, agentic booking launches. <https://9to5google.com/2025/11/17/google-ai-mode-travel/>, 2025. Consultado el 23-09-2026.
- [20] CJ (Commission Junction). Join Booking.com’s travel affiliate programme. <https://www.cj.com/en-gb/publisher/partners/booking.com>, 2026. Consultado el 23-09-2026.
- [21] Expedia Group. Affiliate program. <https://partner.expediagroup.com/en-us/solutions/explore-our-affiliate-program>, 2026. Consultado el 23-09-2026.
- [22] Viator. The premier travel affiliate program – Viator partner resource center. <https://partnerresources.viator.com/>, 2026. Consultado el 23-09-2026.
- [23] Travelpayouts. Kiwi.com affiliate program. <https://www.travelpayouts.com/en/offers/kiwi-affiliate-program/>, 2026. Consultado el 23-09-2026.

- [24] Omio. The Omio affiliate programme. <https://www.omio.com/affiliate>, 2026. Consultado el 23-09-2026.
- [25] Anthropic. Pricing – Claude platform docs. <https://platform.claude.com/docs/en/about-claude/pricing>, 2026. Consultado el 23-09-2026.
- [26] Amazon Web Services. Amazon Bedrock pricing. <https://aws.amazon.com/bedrock/pricing/>, 2026. Consultado el 23-09-2026.
- [27] Insert Affiliate. Mobile app user acquisition cost benchmarks by category. <https://insertaffiliate.com/blog/mobile-app-user-acquisition-cost-benchmarks/>, 2026. Consultado el 23-09-2026.
- [28] Future of Life Institute. The EU AI Act's transparency rules: A practical guide to article 50. <https://artificialintelligenceact.eu/transparency-rules-article-50/>, 2026. Consultado el 23-09-2026.
- [29] Wikivoyage. Wikivoyage:copyleft. <https://en.wikivoyage.org/wiki/Wikivoyage:Copyleft>, 2026. Consultado el 23-09-2026.
- [30] OpenStreetMap Foundation. Copyright and licence (ODbL 1.0). <https://www.openstreetmap.org/copyright>, 2026. Consultado el 23-09-2026.
- [31] Open-Meteo. Terms & privacy. <https://open-meteo.com/en/terms>, 2026. Consultado el 23-09-2026.
- [32] Jefatura del Estado. Ley 9/2017, de 8 de noviembre, de Contratos del Sector Público (art. 118, contratos menores). <https://www.boe.es/buscar/act.php?id=BOE-A-2017-12902>, 2017. Consultado el 23-09-2026.
- [33] Ministerio de Asuntos Económicos y Transformación Digital. Real Decreto 311/2022, de 3 de mayo, por el que se regula el Esquema Nacional de Seguridad. <https://www.boe.es/buscar/act.php?id=BOE-A-2022-7191>, 2022. Consultado el 23-09-2026.
- [34] Ministerio de Política Territorial y Función Pública. Real Decreto 1112/2018, sobre accesibilidad de los sitios web y aplicaciones para dispositivos móviles del sector público. <https://www.boe.es/buscar/act.php?id=BOE-A-2018-12699>, 2018. Consultado el 23-09-2026.
- [35] ETSI. EN 301 549: Accessibility requirements for ICT products and services. <https://www.etsi.org/human-factors-accessibility/en-301-549-v3-the-harmonized-european-standard-for-ict-accessibility>, 2021. Consultado el 23-09-2026.
- [36] SEGITTUR. Red de destinos turísticos inteligentes. <https://www.segittur.es/destinos-turisticos-inteligentes/proyectos-destinos/red-dti/>, 2026. Consultado el 23-09-2026.
- [37] Sebastián Ramírez. FastAPI documentation. <https://fastapi.tiangolo.com/>. Consultado el 23-09-2026.
- [38] Alistair Cockburn. Hexagonal architecture (ports and adapters). <https://alistair.cockburn.us/hexagonal-architecture/>, 2005. Consultado el 23-09-2026.
- [39] M. Jones, J. Bradley, and N. Sakimura. JSON Web Token (JWT). RFC 7519, IETF, 2015. URL <https://www.rfc-editor.org/rfc/rfc7519>.
- [40] M. Jones. JSON Web Key (JWK). RFC 7517, IETF, 2015. URL <https://www.rfc-editor.org/rfc/rfc7517>.
- [41] Amazon Web Services. Best practices for designing and architecting with DynamoDB. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/best-practices.html>. Consultado el 23-09-2026.
- [42] Pydantic. Pydantic documentation: JSON schema. https://docs.pydantic.dev/latest/concepts/json_schema/. Consultado el 23-09-2026.
- [43] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, pages 9459–9474, 2020.
- [44] Amazon Web Services. Amazon titan text embeddings models. <https://docs.aws.amazon.com/bedrock/latest/userguide/titan-embedding-models.html>, 2026. Consultado el 23-09-2026.
- [45] Amazon Web Services. Working with S3 vectors and vector buckets. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-vectors.html>, 2026. Consultado el 23-09-2026.
- [46] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [47] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009.
- [48] Qdrant. Qdrant documentation. <https://qdrant.tech/documentation/>, 2026. Consultado el 23-09-2026.
- [49] Amazon Web Services. Carry out a conversation with the Converse API operations (Amazon Bedrock). <https://docs.aws.amazon.com/bedrock/latest/userguide/conversation-inference.html>, 2026. Consultado el 23-09-2026.
- [50] Amazon Web Services. AWS Lambda Web Adapter. <https://github.com/aws-labs/aws-lambda-web-adapter>, 2026. Consultado el 23-09-2026.
- [51] Amazon Web Services. Amazon S3 Vectors: documentación del usuario. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-vectors.html>, 2026. Consultado el 23-09-2026.
- [52] Amazon Web Services. AWS Pricing Calculator y páginas de precios de Lambda, API Gateway, DynamoDB y Bedrock. <https://calculator.aws/>, 2026. Consultado el 23-09-2026.
- [53] Development Containers. Development container specification. <https://containers.dev/implementors/spec/>, 2026. Consultado el 23-09-2026.
- [54] GitHub. Openid connect in github actions. <https://docs.github.com/en/actions/concepts/security/openid-connect>, 2026. Consultado el 23-09-2026.
- [55] Google. crane: a tool for interacting with remote images and registries. <https://github.com/google/go-containerregistry/tree/main/cmd/crane>, 2026. Consultado el 23-09-2026.
- [56] Anthropic. Claude code documentation. <https://docs.anthropic.com/en/docs/claude-code/overview>, 2026. Consultado el 23-09-2026.
- [57] Conventional Commits. Conventional commits 1.0.0. <https://www.conventionalcommits.org/en/v1.0.0/>, 2019. Consultado el 23-09-2026.
- [58] Michael Nygard. Documenting architecture decisions. <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>, 2011. Consultado el 23-09-2026.
- [59] Amazon Web Services. AWS Well-Architected Framework: pilares de fiabilidad y optimización de costes. <https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html>, 2026. Consultado el 23-09-2026.
- [60] Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016. Disponible en <https://sre.google/sre-book/table-of-contents/>.
- [61] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.
- [62] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, et al. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. <https://arxiv.org/abs/2302.12173>, 2023. arXiv:2302.12173.
- [63] Amazon Web Services. Call a tool with the Amazon Bedrock Converse API. <https://docs.aws.amazon.com/bedrock/latest/userguide/tool-use.html>, 2026. Consultado el 23-09-2026.
- [64] Amazon Web Services. AWS Budgets: managing your costs with budgets. <https://docs.aws.amazon.com/cost-management/latest/userguide/budgets-managing-costs.html>, 2026. Consultado el 23-09-2026.
- [65] Amazon Web Services. Throttle requests to your REST APIs; usage plans and API keys in API Gateway. <https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-request-throttling.html>, 2026. Consultado el 23-09-2026.
- [66] Amazon Web Services. Supported models for reranking in Amazon Bedrock (Cohere Rerank, Amazon Rerank). <https://docs.aws.amazon.com/bedrock/latest/userguide/rerank-supported.html>, 2026. Consultado el 23-09-2026.
- [67] Grafana Labs. Grafana k6: load testing for engineering teams. <https://grafana.com/docs/k6/latest/>. Consultado el 23-09-2026.
- [68] Locust. Locust: an open source load testing tool. <https://locust.io/>. Consultado el 23-09-2026.
- [69] Deque Systems. axe-core: accessibility engine for automated Web UI testing. <https://github.com/dequelabs/axe-core>, 2026. Consultado el 23-09-2026.
- [70] Amazon Web Services. Deployments on an AWS Lambda compute platform (CodeDeploy, canary y linear). <https://docs.aws.amazon.com/codedeploy/latest/userguide/deployment-steps-lambda.html>, 2026. Consultado el 23-09-2026.
- [71] Open Source Routing Machine (OSRM). <https://project-osrm.org/>, 2026. Consultado el 23-09-2026.
- [72] Valhalla: open source routing engine for OpenStreetMap. <https://github.com/valhalla/valhalla>, 2026. Consultado el 23-09-2026.

A Glosario y referencia de decisiones

Glosario.

- ADR** *Architecture Decision Record*: documento numerado con contexto, decisión y consecuencias; no se borra, se sustituye.
- Base de datos vectorial** almacén que indexa *embeddings* y devuelve los más cercanos a una consulta (aquí, S3 Vectors).
- Brief** especificación escrita antes del código: resultado, decisiones cerradas, ficheros, pruebas y aceptación.
- Cold start** arranque en frío: latencia extra de la primera invocación de una Lambda recién creada.
- DMO** *Destination Management Organisation*: organismo público o mixto que promociona un destino.
- Embedding** vector numérico que representa el significado de un texto; textos parecidos quedan cerca.
- Gate de readiness** comprobación automática que decide si el corpus de una ciudad está completo para publicarse.
- Grounding** anclaje: toda tarjeta o lugar propuesto por el modelo debe existir en el corpus recuperado.
- IaC** infraestructura como código: los recursos de AWS se declaran en Terraform y se versionan en Git.
- MRR** *Mean Reciprocal Rank*: media de $1/p$, con p la posición del primer resultado correcto.
- OIDC** OpenID Connect: federación con la que GitHub Actions obtiene credenciales temporales de AWS sin claves.
- RAG** generación aumentada por recuperación: el modelo responde con documentos recuperados en el *prompt*.

Recall@k fracción de consultas con al menos un documento esperado entre los k primeros.

RRF / BM25 BM25: ranking léxico por términos; RRF: fusión de rankings por suma de $1/(k + p)$.

SSE *Server-Sent Events*: flujo HTTP unidireccional de eventos del servidor al navegador.

Tabla única diseño de DynamoDB con todas las entidades en una tabla, distinguidas por claves compuestas.

Turno / traza turno: una petición del usuario y su respuesta; traza: su registro con pasos, tokens, coste y latencia.

Worktree copia de trabajo adicional de Git sobre otra rama, aislada de la principal.

Decisiones de arquitectura (`docs/architecture/adr/`; la 0012 no existe). A: aceptada; S: sustituida; P: en parte.

ADR	Decisión	Estado
0001	Backend en <code>core_api</code> y <code>ai_api</code>	A
0002	JWT sin estado; reenvío del token	S por 0009
0003	Frontend con dos URL base	A
0004	<code>src/</code> , <code>infra/</code> , <code>docs/</code>	A
0005	Trip raíz de agregado	A (S-P 0019)
0006	Modelo de vista en el frontend	A (S-P 0019)
0007	AWS; SSO local, OIDC en CI	A
0008	AWS v2: CloudFront, API GW, Fargate	S por 0009
0009	AWS v3: Lambda, Cognito, 30 €/mes	A (datos: 0023)
0010	Zona DNS y certificado en Terraform	A
0011	Viajes reales, carga en cliente	A (S-P 0019)
0013	Conversaciones en <code>core_api</code>	A
0014	Vectores en S3 Vectors	A
0015	Planificador SSE v2 sin estado	A
0016	Mapa MapLibre + OpenFreeMap	A
0017	Intro y foto de ciudad en el manifiesto	A
0018	Respuestas de chat con tarjetas	A
0019	Los viajes viven en el planificador	A (enm. 0020)
0020	Inicio con sesión: página de viajes	A
0021	Vista previa del sitio del local	A
0022	Foto de hotel resuelta al construir	A
0023	DynamoDB sustituye a RDS	A (enm. 0024)
0024	Trazas de turno y acceso admin	A

Issues citadas. TRA-28 despliegue en GitHub Pages (`.nojekyll`); TRA-133 imágenes a ECR con `crane`; TRA-146 borrador del planificador como viaje; TRA-148 banco de evaluación de Budapest; TRA-151 *spike* Qdrant frente a S3 Vectors; TRA-180 WebGL en el devcontainer; TRA-187 planificador en el *viewport* móvil; TRA-195 dependencias, tres mayores retenidas; TRA-210 corpus curado con cuotas y fotos; TRA-219 retirada de RDS en dos `apply`; TRA-204 Miami a través del pipeline; TRA-232 tokens de *embeddings* en la traza.